```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.G04.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S04_Main";
    }
    /// &lt;summary&gt;
    /// S00_Boot scene host: register mock services -&gt; run seven steps -&gt; switch to main scene.
    /// On failure it stays on S00_Boot and logs the error (no dead-end while loops).
    /// &lt;/summary&gt;
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            // T017 art wiring: the boot beat shows the workshop face from the very
            // first rendered frame instead of a bare camera wash (null-safe: no
            // catalog art = the procedural boot stays exactly as before). The
            // backdrop dies with the scene on the S04 switch - the host builds
            // its own there through the same helper.
            ScrewTapHost.EnsureWorkshopBackdrop();
            // B654: the boot beat carries the same AIGC disclosure footer from the
            // very first rendered frame (first-frame face parity with the S04 host).
            ScrewTapHost.EnsureAigcLabel();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null &amp;&amp; !op.isDone) yield return null;
        }
    }
```

```
}
// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.G04.Boot
{
    /// &lt;summary&gt;
    /// Seven-step boot sequence: Logger -&gt; Time -&gt; Config -&gt; Save -&gt; Locale -&gt; Ads.Preload -&gt; SwitchM
    /// each step logs "[BOOT] n/7 name ok|FAIL". A step exception never blocks later steps.
    /// &lt;/summary&gt;
    public sealed class BootSequence
    {
        public const int TotalSteps = 7;
        readonly Func&lt;bool&gt; _switchScene;
        readonly List&lt;string&gt; _lines = new();
        public BootSequence(Func&lt;bool&gt; switchScene)
        {
            _switchScene = switchScene;
        }
        public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
        public bool RunAll()
        {
            bool ok = true;
            ok &amp;= Run(1, "Logger", LoggerStep);
            ok &amp;= Run(2, "Time", TimeStep);
            ok &amp;= Run(3, "Config", ConfigStep);
            ok &amp;= Run(4, "Save", SaveStep);
            ok &amp;= Run(5, "ILocale", LocaleStep);
            ok &amp;= Run(6, "Ads.Preload", AdsStep);
            ok &amp;= Run(7, "SwitchMain", () =&gt; _switchScene != null &amp;&amp; _switchScene());
            return ok;
        }
        bool Run(int n, string name, Func&lt;bool&gt; step)
        {
            bool result;
            try
            {
                result = step();
            }
            catch (Exception e)
            {
                Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
                result = false;
            }
            string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
            _lines.Add(line);
            Logger.Info("BOOT", line);
            return result;
```

```
        }
        static bool LoggerStep() =&gt; true; // static facade self-initializes on first call
        static bool TimeStep()
        {
            var time = ServiceLocator.Get&lt;ITimeService&gt;();
            return time.ServerNowMs() &gt; 0 &amp;&amp; !time.TimeCheated();
        }
        static bool ConfigStep()
        {
            var config = ServiceLocator.Get&lt;IConfigService&gt;();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock = no hot update
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get&lt;ISaveService&gt;();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get&lt;ILocaleService&gt;();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont);
        }
        static bool AdsStep()
        {
            ServiceLocator.Get&lt;IAdsService&gt;().Preload("boot_warmup");
            return true;
        }
    }
}
// BootServices.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.G04.Boot
{
    /// &lt;summary&gt;
    /// Boot service set (S2-T012 suite wiring). Editor/SY_MOCK builds resolve the
    /// mock platform suite; SY_WECHAT/SY_DOUYIN builds resolve the channel suite -
    /// ads stay category-capped on the shared AdCap in every channel (zero-server).
    /// &lt;/summary&gt;
    public static class BootServices
    {
        /// &lt;summary&gt;Shared ad-cap instance (§  category caps, T008): the magnet host
        /// pre-checks rescue budgets on the same policy the registered ads service
        /// consumes from, so precheck and consume can never disagree.&lt;/summary&gt;
```

```
    public static readonly AdCapPolicy AdCap = new AdCapPolicy();
    /// &lt;summary&gt;Resolved channel suite (S2-T012). Non-null once RegisterAll ran;
    /// later slices (share loop, storage) consume services from this suite.&lt;/summary&gt;
    public static PlatformSuite Suite { get; private set; }
    public static void RegisterAll()
    {
        if (ServiceLocator.TryGet&lt;ITimeService&gt;(out _)) return; // idempotent: double register would throw
        ServiceLocator.Register&lt;ITimeService&gt;(new MockTimeService());
        ServiceLocator.Register&lt;IConfigService&gt;(new MockConfigService());
        ServiceLocator.Register&lt;ISaveService&gt;(new MockSaveService());
        ServiceLocator.Register&lt;ILocaleService&gt;(new MockLocaleService());
        // S2-T012: channel suite wiring. PlatformBootstrap gates SY_WECHAT /
        // SY_DOUYIN builds to the real channel suites; editor/batchmode stays
        // mock. Ads consume from the shared AdCap in every channel so the
        // pre-check/consume same-instance law survives the real-suite switch.
        Suite = PlatformBootstrap.Create(AdCap);
        ServiceLocator.Register&lt;PlatformSuite&gt;(Suite);
        ServiceLocator.Register&lt;IAdsService&gt;(Suite.Ads);
        Logger.Info("BOOT", "services registered (suite=" + PlatformBootstrap.SuiteIdFor(Suite.Channel) + ")");
    }
}
public sealed class MockTimeService : ITimeService
{
    public TimeSource Source =&gt; TimeSource.Mock;
    public long ClientNowMs() =&gt; DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// &lt;summary&gt;SY_MOCK: device time doubles as server time until the real chain lands.&lt;/summary&gt;
    public long ServerNowMs() =&gt; ClientNowMs();
    public bool TimeCheated() =&gt; Math.Abs(ServerNowMs() - ClientNowMs()) &gt; 120_000;
}
public sealed class MockConfigService : IConfigService
{
    public T GetTable&lt;T&gt;(string tableId) where T : class
        =&gt; throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable&lt;T&gt;(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        =&gt; new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) =&gt; "mock-0.0.1";
}
public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) =&gt; PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) =&gt; PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// &lt;summary&gt;No cloud: local storage is the authority (zero-server doctrine).&lt;/summary&gt;
    public SyncResult CloudSync() =&gt; new SyncResult(false, null);
```

```
    }
    public sealed class MockLocaleService : ILocaleService
    {
        public void SetLang(string lang) { }
        /// &lt;summary&gt;Missing key falls back to the key itself with a warning.&lt;/summary&gt;
        public string GetText(string key, params string[] args)
        {
            Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
            return key;
        }
        // U184 real-device CJK face (B569 chain step 3, B657 wiring): LegacyRuntime.ttf
        // ships no zh glyphs, so the zh-CN data faces (StoreTitles_zhCN.json lines)
        // render blank on real devices. The B569 fleet-cast NotoSansSC subset (201KB,
        // cmap-verified missing=0 over the fleet union incl. the ScrewOut 149-char
        // census, FontGate PASS 149/149) rides Resources as NotoSansSC-CrazySubset.otf;
        // future view faces resolve this name and degrade to LegacyRuntime on a miss.
        public FontSet GetFontSet()
            =&gt; new FontSet("NotoSansSC-CrazySubset.otf", "NotoSansSC-CrazySubset.otf", "NotoSansSC-CrazySubset.otf");
    }
}
// BoardModel.cs
// BoardModel.cs - pure C# board data layer for layered screw boards (G04-T004).
// No UnityEngine dependency: the headless BoardDataCheck drives it deterministically
// and the view layer only reads state. Grid cells are integer coordinates per board.
// Slide semantics ported from the E4 graybox sim: a board slides open when its freed
// count reaches slideAt, carrying away the screws still on it and uncovering the board above.
using System;
using System.Collections.Generic;
namespace Sy.G04
{
    /// &lt;summary&gt;One screw in a board slot. State fields feed T005-T008 (tap feel, cascade, reverse thread, magnet
    public sealed class ScrewData
    {
        public int id;
        public int layer;
        public int cellX;
        public int cellY;
        public bool reverse;        // reverse thread: the naive direction tightens instead of loosening
        public bool rusted;         // jammed: untappable, magnet rescue only
        public bool key;            // structural anchor: pulling it topples the whole board (T006)
        public int neededTaps = 1;  // correct taps required to free it
        public int progress;        // correct taps so far
        public int wrongTaps;        // wrong-direction taps so far (reverse teaching stats, T005/T007)
        public bool freed;
        public bool covered;        // a board below this one has not slid away yet
    }
    /// &lt;summary&gt;One stacked board: a grid of slots plus its slide-away state.&lt;/summary&gt;
    public sealed class BoardLayerData
    {
        public int layer;
```

```
    public int width;
    public int height;
    public int slideAt;         // freed screws needed before this board slides open
    public int freed;
    public int occupied;
    public bool slid;
    public int TotalSlots { get { return width * height; } }
}
/// &lt;summary&gt;
/// Multi-board stack: place screws, query slots, free screws, slide boards.
/// Stacking rule: screws on layer N are covered (untappable) until layer N-1 slides.
/// A slide frees the screws still sitting on that board (they tumble away with it).
/// &lt;/summary&gt;
public sealed class BoardStack
{
    public readonly List&lt;BoardLayerData&gt; Boards = new List&lt;BoardLayerData&gt;();
    public readonly List&lt;ScrewData&gt; Screws = new List&lt;ScrewData&gt;();
    public int FreedTotal;
    public int SlidBoards;
    public string LastError;
    public BoardLayerData AddBoard(int width, int height, int slideAt)
    {
        var b = new BoardLayerData
        {
            layer = Boards.Count,
            width = width,
            height = height,
            slideAt = slideAt
        };
        Boards.Add(b);
        return b;
    }
    /// &lt;summary&gt;Places a screw into a free slot. Returns null and sets LastError when the layer, cell, or slo
    public ScrewData AddScrew(int layer, int cellX, int cellY, bool reverse, bool rusted, int neededTaps)
    {
        LastError = null;
        if (layer &lt; 0 || layer &gt;= Boards.Count) { LastError = "layer_out_of_range"; return null; }
        BoardLayerData b = Boards[layer];
        if (cellX &lt; 0 || cellX &gt;= b.width || cellY &lt; 0 || cellY &gt;= b.height) { LastError = "cell_out_of_
        if (ScrewAt(layer, cellX, cellY) != null) { LastError = "slot_occupied"; return null; }
        var s = new ScrewData
        {
            id = Screws.Count,
            layer = layer,
            cellX = cellX,
            cellY = cellY,
            reverse = reverse,
            rusted = rusted,
            neededTaps = Math.Max(1, neededTaps),
            covered = layer &gt; 0 &amp;&amp; !Boards[layer - 1].slid
```

```
        };
        Screws.Add(s);
        b.occupied++;
        return s;
    }
    public ScrewData ScrewAt(int layer, int cellX, int cellY)
    {
        for (int i = 0; i &lt; Screws.Count; i++)
        {
            ScrewData s = Screws[i];
            if (s.layer == layer &amp;&amp; s.cellX == cellX &amp;&amp; s.cellY == cellY) return s;
        }
        return null;
    }
    public bool IsCellFree(int layer, int cellX, int cellY) { return ScrewAt(layer, cellX, cellY) == null; }
    public List&lt;ScrewData&gt; ScrewsOnLayer(int layer)
    {
        var list = new List&lt;ScrewData&gt;();
        for (int i = 0; i &lt; Screws.Count; i++) if (Screws[i].layer == layer) list.Add(Screws[i]);
        return list;
    }
    public int Remaining
    {
        get
        {
            int n = 0;
            for (int i = 0; i &lt; Screws.Count; i++) if (!Screws[i].freed) n++;
            return n;
        }
    }
    /// &lt;summary&gt;Frees a screw; may slide its board (freeing the screws still on it and uncovering the board a
    public bool FreeScrew(ScrewData s)
    {
        if (s == null || s.freed) return false;
        s.freed = true;
        FreedTotal++;
        BoardLayerData b = Boards[s.layer];
        b.freed++;
        if (b.freed &gt;= b.slideAt &amp;&amp; !b.slid) SlideBoard(b.layer);
        return true;
    }
    /// &lt;summary&gt;
    /// Collapses the whole board at once, house-of-cards style (T006): every screw still
    /// on it tumbles away and the board beneath is uncovered. Returns how many tumbled.
    /// &lt;/summary&gt;
    public int ToppleBoard(int layer)
    {
        if (layer &lt; 0 || layer &gt;= Boards.Count) return 0;
        BoardLayerData b = Boards[layer];
        if (b.slid) return 0;
```

```
            int tumbled = 0;
            for (int i = 0; i &lt; Screws.Count; i++)
            {
                ScrewData s = Screws[i];
                if (s.layer == layer &amp;&amp; !s.freed) { s.freed = true; FreedTotal++; tumbled++; }
            }
            b.freed = b.occupied;
            SlideBoard(layer);
            return tumbled;
        }
        void SlideBoard(int layer)
        {
            BoardLayerData b = Boards[layer];
            b.slid = true;
            SlidBoards++;
            for (int i = 0; i &lt; Screws.Count; i++)
            {
                ScrewData s = Screws[i];
                if (s.layer == layer &amp;&amp; !s.freed) { s.freed = true; FreedTotal++; }
                else if (s.layer == layer + 1) s.covered = false;
            }
        }
    }
}
// BoardView.cs
// BoardView.cs - procedural placeholder rendering for the board stack (G04-T004).
// X027 wiring rule: use ArtCatalog art when present, otherwise fall back to procedural
// color blocks; both paths are null-safe so headless gates never break on missing art.
// Palette follows the G04 visual sheet: warm walnut boards, brass normal screws,
// cold-iron reverse screws, rust-red jammed screws, copper-emerald accent.
using UnityEngine;
using Sy.Core;
namespace Sy.G04
{
    /// &lt;summary&gt;Builds placeholder visuals for a BoardStack. Call Rebuild after any state change.&lt;/summary&gt;
    public sealed class BoardView
    {
        static readonly Color[] LayerColors =
        {
            Hex(0x6B4A2F),  // walnut, layer 0
            Hex(0x7A5535),  // lighter walnut, layer 1
            Hex(0x8A6238),  // lightest walnut, layer 2
        };
        const float CellSize = 1f;
        const float LayerStep = 0.22f;   // visual stack offset per board layer
        readonly Transform root;
        static Sprite solid;
        static Sprite circle;
        static Sprite threadCW;
        static Sprite threadCCW;
```

```
static readonly Color MarkNormal = Hex(0x3A2A14);   // dark walnut stripes on brass: subtle
static readonly Color MarkReverse = Hex(0x40BFA3);  // copper-emerald accent on cold iron: pops
public BoardView(Transform root) { this.root = root; }
/// &lt;summary&gt;Stripe sprite for a thread direction (T007 visibility language): the two
/// directions lean opposite ways so the player can read the twist before the first tap.&lt;/summary&gt;
public static Sprite ThreadMarkSprite(bool reverse) { return reverse ? ThreadCCW : ThreadCW; }
public static Color ThreadMarkColor(bool reverse) { return reverse ? MarkReverse : MarkNormal; }
/// &lt;summary&gt;Rebuilds all visuals from the stack. Returns the number of objects created.&lt;/summary&gt;
public int Rebuild(BoardStack stack)
{
    Clear();
    int built = 0;
    // Sorting law (occlusion fix): boards keep the per-layer ladder below,
    // but every screw renders above every plate so the tappable layer reads
    // on the opening beat and covered screws show as the 0.45 ghost the
    // covered flag promises (the old layer*10+5 law buried lower-layer
    // screws behind the same-size boards stacked over them).
    int boardTop = stack.Boards.Count * 10;
    for (int i = 0; i &lt; stack.Boards.Count; i++)
    {
        BoardLayerData b = stack.Boards[i];
        if (b.slid) continue;   // a slid board has visibly left the field (T005 slide feedback)
        Sprite art = ArtRuntime.Get(BoardStem(b.layer));
        var go = MakeSpriteObject(
            art != null ? art : Solid,
            LayerColors[i % LayerColors.Length],
            "board_" + b.layer,
            StackOffset(i),
            new Vector3(b.width * CellSize, b.height * CellSize, 1f),
            i * 10);
        if (go != null) built++;
    }
    for (int i = 0; i &lt; stack.Screws.Count; i++)
    {
        ScrewData s = stack.Screws[i];
        if (s.freed) { built += BuildHole(s, stack) ? 1 : 0; continue; }
        built += BuildScrew(s, stack, boardTop) ? 1 : 0;
    }
    return built;
}
bool BuildScrew(ScrewData s, BoardStack stack, int boardTop)
{
    BoardLayerData b = stack != null &amp;&amp; s.layer &gt;= 0 &amp;&amp; s.layer &lt; stack.Boards.Count ? sta
    if (b == null) return false;
    // Art stems match the S2.5 art sheet (G04-A001..A016); a key screw takes
    // precedence so the topple trigger always reads as the golden emerald piece.
    string stem = s.key ? "screw_key_gold"
        : (s.rusted ? "screw_rusted" : (s.reverse ? "screw_reverse" : "screw_brass_std"));
    Sprite art = ArtRuntime.Get(stem);
    Color color = s.rusted ? Hex(0xA8442A) : (s.reverse ? Hex(0x5C7A99) : Hex(0xB8860B));
```

```
    if (s.covered) color.a = 0.45f;
    // Uncovered screws outrank covered ghosts so the active tap target
    // always paints over the ghosts where layers overlap on screen.
    int order = s.covered ? boardTop + 1 + s.layer : boardTop + 21 + s.layer;
    var go = MakeSpriteObject(
        art != null ? art : Circle,
        color,
        "screw_" + s.id,
        ScrewLocalPos(s, stack),
        new Vector3(0.55f, 0.55f, 1f),
        order);
    if (go != null) BuildThreadMark(go, s, order);
    return go != null;
}
/// &lt;summary&gt;Empty-slot hole for a freed screw (T017 art wiring): the freed
/// cell reads as a hole in the board instead of vanishing. Art-driven
/// enhancement - a null lookup keeps the pre-wiring procedural state (no
/// hole rendered), so headless gates stay byte-identical until the catalog
/// resolves. Idempotent per screw id.&lt;/summary&gt;
public bool BuildHole(ScrewData s, BoardStack stack)
{
    if (s == null || stack == null) return false;
    if (FindChild("hole_" + s.id) != null) return false;   // idempotent
    BoardLayerData b = s.layer &gt;= 0 &amp;&amp; s.layer &lt; stack.Boards.Count ? stack.Boards[s.layer] : null
    if (b == null) return false;
    Sprite art = ArtRuntime.Get("slot_empty_hole");
    if (art == null) return false;   // X027: no art = keep the pre-wiring visual
    var go = MakeSpriteObject(
        art,
        Color.white,          // authored art carries its own hole shading
        "hole_" + s.id,
        ScrewLocalPos(s, stack),
        new Vector3(0.55f, 0.55f, 1f),
        s.layer * 10 + 5);     // hole hugs its own plate, under every screw
    return go != null;
}
/// &lt;summary&gt;Thread-direction mark on every screw head (T007, E3-4 visibility):
/// diagonal stripes lean with the thread - normal CW leans one way in dark walnut,
/// reverse CCW leans the other in the accent color. Drawn above the screw.&lt;/summary&gt;
static void BuildThreadMark(GameObject screwGo, ScrewData s, int screwOrder)
{
    var mark = new GameObject("thread_mark");
    mark.transform.SetParent(screwGo.transform, false);
    var sr = mark.AddComponent&lt;SpriteRenderer&gt;();
    sr.sprite = ThreadMarkSprite(s.reverse);
    sr.color = ThreadMarkColor(s.reverse);
    if (s.covered) { Color c = sr.color; c.a = 0.45f; sr.color = c; }
    sr.sortingOrder = screwOrder + 1;
    mark.transform.localScale = new Vector3(0.42f, 0.42f, 1f);
}
```

```
/// &lt;summary&gt;Catalog stem for a board layer (art sheet G04-A001..A003): layer 0 is
/// the bottom of the stack and carries the darkest tone, matching the sheet ladder.&lt;/summary&gt;
static string BoardStem(int layer)
{
    switch (layer)
    {
        case 0: return "board_walnut_bottom";
        case 1: return "board_walnut_mid";
        default: return "board_walnut_top";
    }
}
/// &lt;summary&gt;World position of a screw cell - shared by rendering and the tap hit test.&lt;/summary&gt;
public static Vector3 ScrewLocalPos(ScrewData s, BoardStack stack)
{
    BoardLayerData b = stack.Boards[s.layer];
    float cx = (s.cellX - (b.width - 1) * 0.5f) * CellSize;
    float cy = (s.cellY - (b.height - 1) * 0.5f) * CellSize;
    return StackOffset(s.layer) + new Vector3(cx, cy, 0f);
}
/// &lt;summary&gt;Looks up a built child object by name (boards and screws). Null when absent.&lt;/summary&gt;
public GameObject FindChild(string childName)
{
    if (root == null) return null;
    Transform t = root.Find(childName);
    return t == null ? null : t.gameObject;
}
public void Clear()
{
    if (root == null) return;
    for (int i = root.childCount - 1; i &gt;= 0; i--) Object.Destroy(root.GetChild(i).gameObject);
}
static Sprite Solid
{
    get { if (solid == null) solid = MakeSprite(4, false); return solid; }
}
static Sprite Circle
{
    get { if (circle == null) circle = MakeSprite(32, true); return circle; }
}
static Sprite ThreadCW
{
    get { if (threadCW == null) threadCW = MakeThreadSprite(true); return threadCW; }
}
static Sprite ThreadCCW
{
    get { if (threadCCW == null) threadCCW = MakeThreadSprite(false); return threadCCW; }
}
/// &lt;summary&gt;16x16 diagonal stripe sprite: 3 ridges leaning with the thread direction.&lt;/summary&gt;
static Sprite MakeThreadSprite(bool leanRight)
{
```

```
    const int size = 16;
    var tex = new Texture2D(size, size, TextureFormat.RGBA32, false);
    for (int y = 0; y &lt; size; y++)
    {
        for (int x = 0; x &lt; size; x++)
        {
            int d = leanRight ? (x - y) : (y - x);
            bool ridge = ((d % 5) + 5) % 5 == 0;   // C# % keeps sign: normalize first
            bool head = x &gt; 1 &amp;&amp; x &lt; 14 &amp;&amp; y &gt; 1 &amp;&amp; y &lt; 14;   // margin so i
            tex.SetPixel(x, y, ridge &amp;&amp; head ? Color.white : Color.clear);
        }
    }
    tex.Apply();
    return Sprite.Create(tex, new Rect(0, 0, size, size), new Vector2(0.5f, 0.5f), (float)size);
}
static Sprite MakeSprite(int size, bool round)
{
    var tex = new Texture2D(size, size, TextureFormat.RGBA32, false);
    float r = size * 0.5f;
    for (int y = 0; y &lt; size; y++)
    {
        for (int x = 0; x &lt; size; x++)
        {
            float dx = x + 0.5f - r;
            float dy = y + 0.5f - r;
            bool inside = !round || dx * dx + dy * dy &lt;= r * r;
            tex.SetPixel(x, y, inside ? Color.white : Color.clear);
        }
    }
    tex.Apply();
    return Sprite.Create(tex, new Rect(0, 0, size, size), new Vector2(0.5f, 0.5f), (float)size);
}
GameObject MakeSpriteObject(Sprite sprite, Color color, string name, Vector3 localPos, Vector3 scale, int sortin
{
    if (root == null) return null;
    var go = new GameObject(name);
    go.transform.SetParent(root, false);
    go.transform.localPosition = localPos;
    go.transform.localScale = SpriteWorldScale(sprite, scale);
    var sr = go.AddComponent&lt;SpriteRenderer&gt;();
    sr.sprite = sprite;
    sr.color = color;
    sr.sortingOrder = sortingOrder;
    return go;
}
/// &lt;summary&gt;World-size law (T017 slice B2, first post-swap sweep finding):
/// the authored scale vector is the TARGET WORLD SIZE, not a scale factor.
/// The procedural placeholders are natively 1x1 world (4px@ppu4, 32px@ppu32),
/// so the two used to coincide; real catalog art carries its own native
/// size (1024px boards @ 100ppu = 10.24 world), so the scale must divide
```

```
        /// it out - art renders at exactly the world size the placeholder had
        /// (the device-matrix measured-union law caught the 10x blow-up: boards
        /// 40.96x20.48 on a 4x2 cell field, screws 2.82 on a 0.55 hit radius).&lt;/summary&gt;
        static Vector3 SpriteWorldScale(Sprite sprite, Vector3 worldSize)
        {
            if (sprite == null) return worldSize;
            Vector3 s = sprite.bounds.size;
            float x = s.x &gt; 0.0001f ? worldSize.x / s.x : 1f;
            float y = s.y &gt; 0.0001f ? worldSize.y / s.y : 1f;
            return new Vector3(x, y, worldSize.z);
        }
        static Vector3 StackOffset(int layer)
        {
            return new Vector3(layer * LayerStep, layer * LayerStep, 0f);
        }
        static Color Hex(int rgb)
        {
            return new Color(((rgb &gt;&gt; 16) &amp; 0xFF) / 255f, ((rgb &gt;&gt; 8) &amp; 0xFF) / 255f, (rgb &amp; 0xF
        }
    }
}
// CameraFit.cs
// CameraFit.cs - deterministic device-adaptation contract (G04-T023, device matrix).
// Red line 10 portrait lock pairs with this fit: the game column is a frozen
// DESIGN FRAME (the world rect that must stay fully visible inside the SAFE
// area on every device). Apply computes an orthographic camera that maps the
// design frame into the safe rect - notch and rounded corners only ever add
// letterbox, never clip. Numbers come in as data so headless gates can drive
// tier cards (1080x1920 / 720x1280) deterministically; ApplyToMainCamera is
// the runtime face reading Screen. The resting layout is the contract - exit
// animations (PopFx / SlideFx / ToppleFx) intentionally fly content off-frame.
using UnityEngine;
namespace Sy.G04
{
    public static class CameraFit
    {
        /// &lt;summary&gt;Design frame: HUD bands + the resting board stack. Covers the
        /// Caption (y=2.7) / Counter row (y=-2.7) text bands, the DirToggle hit
        /// box (x=-2.35 +- 0.85) and the Hint/Skip stack (x=2.35) with margin.&lt;/summary&gt;
        public static readonly Rect DesignFrame = new Rect(-3.6f, -3.2f, 7.2f, 6.4f);
        /// &lt;summary&gt;Pixel-density floor per tier (px per world unit, safe-rect
        /// basis): 720p low tier must stay readable, 1080p is the reference.&lt;/summary&gt;
        public const float PxPerWuFloor1080 = 140f;
        public const float PxPerWuFloor720 = 90f;
        /// &lt;summary&gt;Result of one fit: the safe rect in world space plus the
        /// pixels-per-world-unit the fit produced (legibility face).&lt;/summary&gt;
        public struct FitResult
        {
            public Rect SafeWorld;
            public float PxPerWu;
```

```
        }
        /// &lt;summary&gt;Fits the camera so the design frame sits fully inside the
        /// safe viewport rect. safeNv is normalized (x, y, w, h in 0..1).&lt;/summary&gt;
        public static FitResult Apply(Camera cam, int screenW, int screenH, Rect safeNv)
        {
            float safeWpx = screenW * safeNv.width;
            float safeHpx = screenH * safeNv.height;
            float safeAspect = safeWpx / safeHpx;
            float designAspect = DesignFrame.width / DesignFrame.height;
            // world units per pixel: bind the wider-relative axis, letterbox the other
            float wpp = designAspect &gt;= safeAspect
                ? DesignFrame.width / safeWpx
                : DesignFrame.height / safeHpx;
            cam.orthographic = true;
            cam.orthographicSize = screenH * 0.5f * wpp;
            float visW = screenW * wpp;
            float visH = screenH * wpp;
            // center the design frame on the safe rect's center
            float cu = safeNv.x + safeNv.width * 0.5f;
            float cv = safeNv.y + safeNv.height * 0.5f;
            float cx = DesignFrame.center.x - (cu - 0.5f) * visW;
            float cy = DesignFrame.center.y - (cv - 0.5f) * visH;
            cam.transform.position = new Vector3(cx, cy, cam.transform.position.z);
            float sx = cx + (safeNv.xMin - 0.5f) * visW;
            float sy = cy + (safeNv.yMin - 0.5f) * visH;
            return new FitResult
            {
                SafeWorld = new Rect(sx, sy, safeWpx * wpp, safeHpx * wpp),
                PxPerWu = 1f / wpp,
            };
        }
        /// &lt;summary&gt;Runtime face: fit the scene's main camera from the real
        /// screen and its safe area (notch/corners compensate by letterbox).
        /// Null-safe and skips on degenerate screen dims so headless gates that
        /// build the host without a camera never break.&lt;/summary&gt;
        public static void ApplyToMainCamera()
        {
            Camera cam = Camera.main;
            if (cam == null) return;
            int w = Screen.width, h = Screen.height;
            if (w &lt;= 0 || h &lt;= 0) return;
            Rect safe = Screen.safeArea;
            Rect safeNv = new Rect(safe.x / w, safe.y / h, safe.width / w, safe.height / h);
            Apply(cam, w, h, safeNv);
        }
    }
}
// ClickSynth.cs
// ClickSynth.cs - procedural placeholder for the twist click hook (G04-T005).
// The two-burst "ka-da" rhythm (high tick then lower tick ~35ms later) is the G04
```

```
// audio hook signature, woodwork-beat family; real assets arrive at S2.5 through the
// official art/audio pipeline and replace this clip (U020 two-tier rule).
using UnityEngine;
namespace Sy.G04
{
    public static class ClickSynth
    {
        public const string ClipName = "screw_click_procedural";
        const int SampleRate = 22050;
        static AudioClip click;
        static AudioSource source;
        public static AudioClip EnsureClip()
        {
            if (click != null) return click;
            const float dur = 0.09f;
            int count = (int)(SampleRate * dur);
            var data = new float[count];
            for (int i = 0; i &lt; count; i++)
            {
                float t = (float)i / SampleRate;
                data[i] = 0.9f * Tick(t, 0.000f, 900f) + 0.7f * Tick(t, 0.035f, 600f);
            }
            click = AudioClip.Create(ClipName, count, 1, SampleRate, false);
            click.SetData(data, 0);
            return click;
        }
        static float Tick(float t, float start, float freq)
        {
            float dt = t - start;
            if (dt &lt; 0f) return 0f;
            return Mathf.Exp(-dt * 90f) * Mathf.Sin(2f * Mathf.PI * freq * dt);
        }
        public static void EnsureSource(Component host)
        {
            if (source != null) return;
            source = host.gameObject.AddComponent&lt;AudioSource&gt;();
            source.playOnAwake = false;
        }
        /// &lt;summary&gt;Plays the click; silently skipped outside play mode so headless checks stay deterministic.&lt
        public static void PlayClick()
        {
            if (source == null || !Application.isPlaying) return;
            if (click == null) EnsureClip();
            source.PlayOneShot(click);
        }
    }
}
// HintFx.cs
// HintFx.cs - hint highlight feedback (G04-T009): the hinted screw breathes in the
// copper-emerald accent and its thread mark pulses along, pointing the player's eye
```

```
// at the next movable screw for a few seconds. Update-driven like TightenFx so
// headless checks can assert presence; destroys only the component - the screw
// stays on the board, fully tappable.
using UnityEngine;
namespace Sy.G04
{
    public sealed class HintFx : MonoBehaviour
    {
        public const float Life = 2.6f;
        public static readonly Color Accent = new Color(0.25f, 0.75f, 0.64f, 1f);   // same accent as the magnet offer
        float age;
        Vector3 originScale;
        Color originColor;
        SpriteRenderer sr;
        void Awake()
        {
            originScale = transform.localScale;
            sr = GetComponent&lt;SpriteRenderer&gt;();
            if (sr != null) originColor = sr.color;
        }
        void Update()
        {
            age += Time.deltaTime;
            float k = age / Life;
            // decaying breath: reads as "this one, right here", not a jitter
            float breath = 1f + 0.13f * (1f - k) * (0.5f + 0.5f * Mathf.Sin(age * 9f));
            transform.localScale = originScale * breath;
            if (sr != null)
                sr.color = Color.Lerp(originColor, Accent, 0.55f * (1f - k) * (0.5f + 0.5f * Mathf.Sin(age * 9f)));
            var mark = transform.Find("thread_mark");
            if (mark != null)
            {
                float pulse = 0.42f * (1f + 0.5f * (1f - k) * (0.5f + 0.5f * Mathf.Sin(age * 9f)));
                mark.localScale = new Vector3(pulse, pulse, 1f);
            }
            if (age &gt;= Life)
            {
                transform.localScale = originScale;
                if (sr != null) sr.color = originColor;
                Destroy(this);
            }
        }
    }
}
// LevelCurve.cs
// LevelCurve.cs - shipped difficulty-curve data layer for G04 (T029 slice 1).
// Productizes the T028 authored harness ramp into the real level sequence:
// 60 shipped levels over 6 ten-level segments. Every segment anchor is
// monotonically trickier than the one before it (pitch-book sec-5 promise as
// measurable data): more boards, denser screws, more reverse threads, scarcer
```

```
// key anchors, heavier organic-jam pressure, tougher key taps.
// Pure C# and fully deterministic: the only draw is a splitmix64-seeded LCG
// over the level index, so the host can replay any level bit-exact and the
// engine itself stays zero-RNG (T028 harness law preserved).
using System;
using System.Collections.Generic;
namespace Sy.G04
{
    /// &lt;summary&gt;Authored difficulty parameters for one shipped level (segment step-ramp).&lt;/summary&gt;
    public sealed class LevelParams
    {
        public int level;            // 1..60
        public int segment;          // 1..6
        public int boardCount;       // stacked boards
        public double screwDensity;  // occupied-slot share of the board grid
        public double reverseRatio;  // reverse-thread share of placed screws
        public double keyRatio;      // key-anchor share (scarcity: lower = trickier)
        public double jamPressure;  // P(level draws an organic jam layer)
        public int maxNeededTaps;    // taps required by the toughest screw
    }
    public static class LevelCurve
    {
        public const int SegmentSize = 10;
        public const int LevelCount = 60;
        // Segment anchors, index 0 = segment 1. Monotone by construction:
        // boards/density/reverse/jam/taps climb, key anchors go scarcer.
        public static readonly int[] SegBoards = { 2, 3, 3, 4, 4, 5 };
        public static readonly double[] SegDensity = { 0.55, 0.62, 0.68, 0.74, 0.80, 0.85 };
        public static readonly double[] SegReverse = { 0.00, 0.05, 0.10, 0.15, 0.20, 0.25 };
        public static readonly double[] SegKey = { 0.20, 0.16, 0.12, 0.08, 0.05, 0.02 };
        public static readonly double[] SegJam = { 0.00, 0.08, 0.14, 0.20, 0.26, 0.32 };
        public static readonly int[] SegTaps = { 1, 1, 1, 2, 2, 2 };
        /// &lt;summary&gt;Deterministic per-level draw stream (splitmix64 over the level index, T028 pattern).&lt;/summ
        struct Lcg
        {
            ulong s;
            public Lcg(int seed) { s = (ulong)(seed * 2654435761u + 40503u) | 1u; }
            public double Next()
            {
                s = s * 6364136223846793005ul + 1442695040888963407ul;
                return (double)(s &gt;&gt; 11) / 9007199254740992.0;
            }
        }
        static Lcg LevelLcg(int level)
        {
            ulong z = 0x9E3779B97F4A7C15ul * (ulong)(uint)level;
            z = (z ^ (z &gt;&gt; 30)) * 0xBF58476D1CE4E5B9ul;
            z = (z ^ (z &gt;&gt; 27)) * 0x94D049BB133111ECul;
            z ^= z &gt;&gt; 31;
            return new Lcg((int)z);
```

```
}
static int Occ(double density) { return (int)Math.Floor(density * 12.0 + 0.5); }
public static LevelParams ParamsFor(int level)
{
    if (level &lt; 1 || level &gt; LevelCount) return null;
    int seg = (level - 1) / SegmentSize + 1;
    return new LevelParams
    {
        level = level,
        segment = seg,
        boardCount = SegBoards[seg - 1],
        screwDensity = SegDensity[seg - 1],
        reverseRatio = SegReverse[seg - 1],
        keyRatio = SegKey[seg - 1],
        jamPressure = SegJam[seg - 1],
        maxNeededTaps = SegTaps[seg - 1]
    };
}
public static List&lt;LevelParams&gt; AllLevels()
{
    var list = new List&lt;LevelParams&gt;();
    for (int lvl = 1; lvl &lt;= LevelCount; lvl++) list.Add(ParamsFor(lvl));
    return list;
}
/// &lt;summary&gt;
/// Builds the shipped board stack for one level. Layer shape mirrors the
/// T028 family: bottom plate, jam-risk plate at layer 1, plain plates above.
/// The organic jam (layer 1, exactly 2 rusted riders, slideAt = occ - 1)
/// is the only face that can hard-stuck a perfect player, and a single
/// magnet yank always clears it - the live 4/day rescue budget stays the
/// honest lose wall. Non-jam levels never deadlock by construction.
/// &lt;/summary&gt;
public static BoardStack BuildLevel(int level)
{
    LevelParams p = ParamsFor(level);
    if (p == null) return null;
    var rng = LevelLcg(level);
    var stack = new BoardStack();
    bool jam = rng.Next() &lt; p.jamPressure;
    int occ = Occ(p.screwDensity);
    int revCount = (int)Math.Floor(p.reverseRatio * occ + 0.5);
    int keyCount = (int)Math.Floor(p.keyRatio * occ + 0.5);
    // When the organic jam fires, the key anchor rides the top board instead of
    // the jam layer: freeing a key on the jam layer topples that very board and
    // sweeps the rusted riders away with it (the house-of-cards cleared the jam
    // for free, so mid-curve jam levels never reached the magnet). On the top
    // board the key still delivers its topple, while the rusted pair stays on
    // the jam layer to the end - the jam binds, the deadlock offer fires, and
    // one magnet yank clears it (1-magnet economy across the whole curve).
    int keyBoard = jam ? p.boardCount - 1 : 1;
```

```
            for (int b = 0; b &lt; p.boardCount; b++)
            {
                bool jamLayer = jam &amp;&amp; b == 1;
                stack.AddBoard(4, 3, Math.Max(1, occ - 1));
                int revPlaced = 0;
                for (int i = 0; i &lt; occ; i++)
                {
                    // stride-5 rotation over the 4x3 grid: distinct cells per layer
                    int cell = (i * 5 + b * 3 + level) % 12;
                    bool key = b == keyBoard &amp;&amp; keyCount &gt; 0 &amp;&amp; i == 0;
                    bool rust = jamLayer &amp;&amp; !key &amp;&amp; i &gt;= occ - 2;   // exactly the last two slots
                    bool rev = !key &amp;&amp; !rust &amp;&amp; revPlaced &lt; revCount &amp;&amp; i % 2 == 1 &amp;&amp;
                    int needed = key ? p.maxNeededTaps : 1;
                    stack.AddScrew(b, cell % 4, cell / 4, rev, rust, needed).key = key;
                }
            }
            return stack;
        }
    }
}
// MilestoneSchedule.cs
// MilestoneSchedule.cs - authored cross-run milestone/goal rows for G04 (T030
// slice 2, U138 F5 law). Career carriers in the save (levels cleared, flawless
// runs, freed screws, rescues) are the momentum face: every authored row names
// one career metric and a target, SaveCheck Section E asserts the schedule
// against the real model, and the claimed bitmask banks finished goals so the
// chase pointer only moves forward. Pure C#, deterministic, host-independent:
// a later live-ops batch may render these rows as goal tiles off the save.
using System;
namespace Sy.G04
{
    public sealed class CareerMilestone
    {
        public int id;         // claim bit index (banked in save.milestonesClaimed)
        public string metric;  // career carrier vocabulary key
        public int target;     // goal completes at metric &gt;= target
        public string title;   // short goal text (the brag line)
        public string note;    // one-line intent
    }
    public static class MilestoneSchedule
    {
        // Metric vocabulary (validated by SaveCheck Section E against the model):
        //   levels     career boards cleared (levelsCleared)
        //   flawless   career zero-jam zero-assist clears (flawlessClears - the
        //              single-run FLAWLESS brag promoted to a cross-run carrier)
        //   freed      lifetime freed screws (totalFreed)
        //   rescues    lifetime magnet rescues (rescuesTotal)
        //   codex      freed-screw codex size (all four books together)
        public static readonly string[] Metrics =
        {
```

```
    "levels", "flawless", "freed", "rescues", "codex",
};
public static readonly CareerMilestone[] Rows =
{
    new CareerMilestone { id = 0,  metric = "levels",   target = 3,   title = "First Three Boards",    note = "s
    new CareerMilestone { id = 1,  metric = "freed",    target = 50,  title = "Fifty Screws Freed",     note = "
    new CareerMilestone { id = 2,  metric = "flawless", target = 1,   title = "First Flawless Run",    note = "t
    new CareerMilestone { id = 3,  metric = "levels",   target = 10,  title = "Ten Boards Down",        note = "
    new CareerMilestone { id = 4,  metric = "rescues",  target = 5,   title = "Career Rescuer",        note = "f
    new CareerMilestone { id = 5,  metric = "codex",     target = 100, title = "Codex Collector",        note =
    new CareerMilestone { id = 6,  metric = "levels",   target = 25,  title = "Quarter Curve",          note = "
    new CareerMilestone { id = 7,  metric = "flawless", target = 10,  title = "Ten Perfect Clears",    note = "t
    new CareerMilestone { id = 8,  metric = "freed",    target = 500, title = "Half-Thousand Screws",  note = "t
    new CareerMilestone { id = 9,  metric = "rescues",  target = 25,  title = "Jam Breaker",            note = "
    new CareerMilestone { id = 10, metric = "levels",   target = 50,  title = "Fifty Boards",          note = "t
    new CareerMilestone { id = 11, metric = "flawless", target = 50,  title = "Flawless Half-Century", note = "c
};
/// &lt;summary&gt;Career carrier value for a metric key (0 when unknown).&lt;/summary&gt;
public static int Progress(G04SaveModel m, string metric)
{
    if (m == null) return 0;
    switch (metric)
    {
        case "levels":   return m.levelsCleared;
        case "flawless": return m.flawlessClears;
        case "freed":    return m.totalFreed;
        case "rescues":  return m.rescuesTotal;
        case "codex":    return m.codexNaive + m.codexCovered + m.codexReverse + m.codexRusted;
        default:         return 0;
    }
}
public static bool Complete(CareerMilestone row, G04SaveModel m)
    =&gt; Progress(m, row.metric) &gt;= row.target;
/// &lt;summary&gt;True when the goal row is banked (claim bit set).&lt;/summary&gt;
public static bool Claimed(G04SaveModel m, CareerMilestone row)
    =&gt; m != null &amp;&amp; (m.milestonesClaimed &amp; (1 &lt;&lt; row.id)) != 0;
/// &lt;summary&gt;First authored row that is not complete yet - the goal the
/// next run chases; null when the whole schedule is done.&lt;/summary&gt;
public static CareerMilestone NextGoal(G04SaveModel m)
{
    for (int i = 0; i &lt; Rows.Length; i++)
        if (!Complete(Rows[i], m)) return Rows[i];
    return null;
}
/// &lt;summary&gt;Bank a finished goal row (sets the claim bit). Momentum only
/// moves forward - a banked row never re-awards.&lt;/summary&gt;
public static void Claim(G04SaveModel m, CareerMilestone row)
{
    if (m == null || row == null) return;
    m.milestonesClaimed |= 1 &lt;&lt; row.id;
```

```
        }
    }
}
// NoveltySchedule.cs
// NoveltySchedule.cs - authored novelty timetable for G04 (T030 slice 1, U138 F4 law).
// One row per segment of the shipped LevelCurve naming the mechanic faces that
// debut (or step up) there - the "what is new in this segment" teach face of the
// curve as data. NoveltyFunCheck asserts every row against the 60 built levels,
// so the authored narrative can never drift from the shipped reality.
// Pure C#, deterministic, host-independent: a later live-ops batch may render
// these keys as new-mechanic captions when the curve wires into ScrewTapHost.
using System;
namespace Sy.G04
{
    public sealed class SegmentNovelty
    {
        public int segment;        // 1..6
        public string[] newFaces; // mechanic faces debuting or stepping up here
        public string note;       // one-line teach intent
    }
    public static class NoveltySchedule
    {
        public const int SegmentCount = 6;
        // Face vocabulary (validated by NoveltyFunCheck against built levels):
        //   core_loop       twist-pop + slide + count-up (the teach segment)
        //   key_anchor      house-of-cards topple anchor first appears
        //   jam_rescue      organic jam layer + magnet rescue economy debuts
        //   reverse         reverse-thread screws first appear (read the mark)
        //   two_tap_key     key anchors start demanding two twists
        //   deep_stack      stacks grow to four boards
        //   reverse_double  two reverse screws per board (mixed pressure)
        //   master_stack    five-board stacks
        //   key_scarcity    key anchors vanish entirely (the mastery exam)
        //   jam_peak        jam pressure at the authored max
        public static readonly SegmentNovelty[] Rows =
        {
            new SegmentNovelty { segment = 1, newFaces = new string[] { "core_loop", "key_anchor" },
                note = "teach the twist-pop-slide loop and the first house-of-cards topple" },
            new SegmentNovelty { segment = 2, newFaces = new string[] { "jam_rescue" },
                note = "organic jam layer debuts - deadlock offer plus the one-magnet economy" },
            new SegmentNovelty { segment = 3, newFaces = new string[] { "reverse" },
                note = "first reverse threads - read the mark, learn the flip" },
            new SegmentNovelty { segment = 4, newFaces = new string[] { "two_tap_key", "deep_stack" },
                note = "keys demand two twists; stacks grow to four boards" },
            new SegmentNovelty { segment = 5, newFaces = new string[] { "reverse_double" },
                note = "mixed pressure - two reverse screws per board, jam pressure 0.26" },
            new SegmentNovelty { segment = 6, newFaces = new string[] { "master_stack", "key_scarcity", "jam_peak" },
                note = "mastery exam - five boards, no key anchors, jam pressure at max" },
        };
    }
```

```
}
// PopFx.cs
// PopFx.cs - screw spin-out-and-fly animation (G04-T005, part of the pop triple).
// Runs purely in Update so it stays inert in edit mode (headless checks can still
// assert the component). Self-destructs when the flight ends.
using UnityEngine;
namespace Sy.G04
{
    public sealed class PopFx : MonoBehaviour
    {
        public const float Life = 0.5f;
        public float delay;          // cascade stagger: carried screws tumble one after another
        public float vx = 2.4f;
        public float vy = 3.4f;
        public float spinDegPerSec = 540f;
        float age;
        SpriteRenderer sr;
        Color color0;
        void Awake()
        {
            sr = GetComponent&lt;SpriteRenderer&gt;();
            if (sr != null) color0 = sr.color;
        }
        void Update()
        {
            if (delay &gt; 0f)
            {
                delay -= Time.deltaTime;
                if (delay &gt; 0f) return;
            }
            age += Time.deltaTime;
            float k = age / Life;
            transform.localPosition += new Vector3(vx, vy, 0f) * Time.deltaTime;
            vy -= 9.8f * Time.deltaTime;   // gravity: a short arc, then it drops off the board
            transform.Rotate(0f, 0f, spinDegPerSec * Time.deltaTime);
            if (sr != null)
            {
                Color c = color0;
                c.a = Mathf.Max(0f, 1f - k);
                sr.color = c;
            }
            if (age &gt;= Life) Destroy(gameObject);
        }
    }
}
// SaveWire.cs
// SaveWire.cs - local-authority save model for the meta loop (G04-T013).
// One flat model: level progress, daily counters keyed on the SAME day key the
// ad cap uses (one boundary flips both), and the freed-screw codex. Local
// storage is the authority - this wire never calls CloudSync (red line 12,
```

```
// zero-server). The Resolver seam exists so SaveCheck can prove that doctrine
// with a counting spy instead of trusting the source shape.
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.G04
{
    /// &lt;summary&gt;Persisted meta progression (flat model, MiniJson round-trip).&lt;/summary&gt;
    public sealed class G04SaveModel
    {
        public int v = 1;              // schema version (migration hook)
        public string dayKey = "";     // AdCapPolicy.TodayKey() the daily counters belong to
        public int levelsCleared;      // boards fully cleared
        public int totalFreed;         // lifetime freed screws
        public int rescuesToday;       // magnet rescues consumed on dayKey
        public int hintsToday;         // hint shows consumed on dayKey
        public int skipsToday;         // skip-the-level takes consumed on dayKey
        public int codexNaive;         // freed screws by flavor (the collection book)
        public int codexCovered;
        public int codexReverse;
        public int codexRusted;
        // T030 slice 2 (U138 F5 law): cross-run career carriers - the momentum face.
        // Additive fields, no version bump: pre-expansion payloads load with these
        // defaulted to zero (the GetInt fallback IS the migration path).
        public int flawlessClears;      // career zero-jam zero-assist clears (the promoted FLAWLESS brag)
        public int rescuesTotal;        // lifetime magnet rescues (the daily take lives in rescuesToday)
        public int milestonesClaimed;   // banked career goals, one bit per MilestoneSchedule row id
    }
    public static class SaveWire
    {
        public const string MainSlot = "g04_main";
        /// &lt;summary&gt;Slot the parameterless host path reads/writes; checks point it
        /// at a probe slot so batch runs stay hermetic against the real save.&lt;/summary&gt;
        public static string ActiveSlot = MainSlot;
        /// &lt;summary&gt;Service seam: default resolves the registered ISaveService;
        /// SaveCheck swaps in a CloudSync-counting spy (a doctrine needs an instrument).&lt;/summary&gt;
        public static Func&lt;ISaveService&gt; Resolver = ResolveRegistered;
        /// &lt;summary&gt;In-memory authority: loaded on host boot, persisted on every beat worth saving.&lt;/summary&g
        public static readonly G04SaveModel State = new G04SaveModel();
        public static void LoadOrDefault() { LoadOrDefault(ActiveSlot); }
        public static void Persist() { Persist(ActiveSlot); }
        public static void LoadOrDefault(string slot)
        {
            ISaveService save = Resolver();
            string json = save == null ? null : save.Load(slot);
            G04SaveModel m = string.IsNullOrEmpty(json) ? null : Deserialize(json);
            CopyInto(m ?? new G04SaveModel(), State);   // missing/corrupt payload =&gt; defaults ("or default" is hones
            DayAlign();
        }
```

```
public static void Persist(string slot)
{
    ISaveService save = Resolver();
    if (save != null) save.Save(slot, Serialize(State));
}
/// &lt;summary&gt;Daily boundary: counters belong to their day key - a stale key
/// means a new day, so the daily counters reset on the SAME key format the
/// ad cap policy flips on. Meta progression (levels/codex) is forever-local.&lt;/summary&gt;
public static void DayAlign()
{
    string today = AdCapPolicy.TodayKey();
    if (State.dayKey == today) return;
    State.dayKey = today;
    State.rescuesToday = 0;
    State.hintsToday = 0;
    State.skipsToday = 0;
}
public static void OnLevelCleared() { OnLevelCleared(false); }   // legacy seam: no verdict info = not flawless
/// &lt;summary&gt;T030: the clear beat carries its flawless verdict - a zero-jam
/// zero-assist clear banks the career carrier (U138 F5, cross-run momentum).&lt;/summary&gt;
public static void OnLevelCleared(bool flawless)
{
    State.levelsCleared++;
    if (flawless) State.flawlessClears++;
    Persist();
}
/// &lt;summary&gt;Codex classification: the jam flavor wins over the thread flavor
/// (a rusted reverse screw is a rusted save - the rescue story dominates).&lt;/summary&gt;
public static void OnFreed(ScrewData s)
{
    State.totalFreed++;
    if (s.rusted) State.codexRusted++;
    else if (s.reverse) State.codexReverse++;
    else if (s.covered) State.codexCovered++;
    else State.codexNaive++;
    Persist();
}
public static void OnAdConsumed(string pointId)
{
    if (pointId == ScrewTapHost.MagnetRescuePoint)
    {
        State.rescuesToday++;
        State.rescuesTotal++;   // T030: the career tally rides the same beat (cross-run)
    }
    else if (pointId == ScrewTapHost.HintPoint) State.hintsToday++;
    else if (pointId == ScrewTapHost.SkipPoint) State.skipsToday++;
    Persist();
}
// ---- MiniJson round-trip ----
public static string Serialize(G04SaveModel m)
```

```
{
    var d = new Dictionary&lt;string, object&gt;
    {
        { "v", (long)m.v },
        { "dayKey", m.dayKey },
        { "levelsCleared", (long)m.levelsCleared },
        { "totalFreed", (long)m.totalFreed },
        { "rescuesToday", (long)m.rescuesToday },
        { "hintsToday", (long)m.hintsToday },
        { "skipsToday", (long)m.skipsToday },
        { "codexNaive", (long)m.codexNaive },
        { "codexCovered", (long)m.codexCovered },
        { "codexReverse", (long)m.codexReverse },
        { "codexRusted", (long)m.codexRusted },
        { "flawlessClears", (long)m.flawlessClears },
        { "rescuesTotal", (long)m.rescuesTotal },
        { "milestonesClaimed", (long)m.milestonesClaimed },
    };
    return MiniJson.Serialize(d);
}
public static G04SaveModel Deserialize(string json)
{
    try
    {
        if (!(MiniJson.Deserialize(json) is Dictionary&lt;string, object&gt; d)) return null;
        var m = new G04SaveModel();
        m.v = GetInt(d, "v", 1);
        m.dayKey = d.TryGetValue("dayKey", out var k) &amp;&amp; k is string s ? s : "";
        m.levelsCleared = GetInt(d, "levelsCleared", 0);
        m.totalFreed = GetInt(d, "totalFreed", 0);
        m.rescuesToday = GetInt(d, "rescuesToday", 0);
        m.hintsToday = GetInt(d, "hintsToday", 0);
        m.skipsToday = GetInt(d, "skipsToday", 0);
        m.codexNaive = GetInt(d, "codexNaive", 0);
        m.codexCovered = GetInt(d, "codexCovered", 0);
        m.codexReverse = GetInt(d, "codexReverse", 0);
        m.codexRusted = GetInt(d, "codexRusted", 0);
        m.flawlessClears = GetInt(d, "flawlessClears", 0);      // T030: pre-expansion payloads
        m.rescuesTotal = GetInt(d, "rescuesTotal", 0);          // default to zero here -
        m.milestonesClaimed = GetInt(d, "milestonesClaimed", 0); // zero save-break (backward-compat law)
        return m;
    }
    catch { return null; }
}
static int GetInt(Dictionary&lt;string, object&gt; d, string key, int fallback)
    =&gt; d.TryGetValue(key, out var o) &amp;&amp; o is long l ? (int)l : fallback;
static void CopyInto(G04SaveModel src, G04SaveModel dst)
{
    dst.v = src.v;
    dst.dayKey = src.dayKey;
```

```
            dst.levelsCleared = src.levelsCleared;
            dst.totalFreed = src.totalFreed;
            dst.rescuesToday = src.rescuesToday;
            dst.hintsToday = src.hintsToday;
            dst.skipsToday = src.skipsToday;
            dst.codexNaive = src.codexNaive;
            dst.codexCovered = src.codexCovered;
            dst.codexReverse = src.codexReverse;
            dst.codexRusted = src.codexRusted;
            dst.flawlessClears = src.flawlessClears;
            dst.rescuesTotal = src.rescuesTotal;
            dst.milestonesClaimed = src.milestonesClaimed;
        }
        static ISaveService ResolveRegistered()
        {
            return ServiceLocator.TryGet&lt;ISaveService&gt;(out var save) ? save : null;
        }
    }
}
// ScrewAudioCatalog.cs
// ScrewAudioCatalog.cs - generated-audio catalog (G04 audio wiring prep,
// batch B643). X027-isomorphic to the G08 CampusAudioCatalog / G13
// FarmAudioCatalog paradigm: the ScrewAudioSetup editor tool scans
// Assets/_Game/Art/Audio and serializes clip references here so runtime code
// can resolve audio by filename stem. The asset lives at
// Assets/_Game/Resources/ScrewAudioCatalog.asset (Resources keeps the
// playtest-slice lookup build-safe with zero Addressables wiring, same
// rationale as ArtCatalog). Missing clip = null = stay silent (X027
// null-safe law): headless gates stay green whatever the import state is -
// the cloudF wave (B-AUDIO-B637, 4 stems pending) lands later and the
// catalog rebuild lights the whole face up with zero code churn.
using System;
using System.Collections.Generic;
using UnityEngine;
namespace Sy.G04
{
    [CreateAssetMenu(fileName = "ScrewAudioCatalog", menuName = "Sy/G04 Screw Audio Catalog")]
    public sealed class ScrewAudioCatalog : ScriptableObject
    {
        [Serializable]
        public sealed class Entry
        {
            public string key;      // filename stem, e.g. "g04_hook_bgm"
            public AudioClip clip; // direct reference keeps builds self-contained
        }
        public List&lt;Entry&gt; entries = new List&lt;Entry&gt;();
        Dictionary&lt;string, AudioClip&gt; _index;
        void OnEnable()
        {
            // Serializer revival can pair a fresh instance with a stale cache;
```

```
            // drop it so the next Get() rebuilds from entries.
            _index = null;
        }
        public AudioClip Get(string key)
        {
            if (string.IsNullOrEmpty(key)) return null;
            if (_index == null)
            {
                _index = new Dictionary&lt;string, AudioClip&gt;(entries.Count, StringComparer.Ordinal);
                for (int i = 0; i &lt; entries.Count; i++)
                {
                    var e = entries[i];
                    if (e != null &amp;&amp; !string.IsNullOrEmpty(e.key) &amp;&amp; e.clip != null &amp;&amp; !_index.C
                        _index[e.key] = e.clip;
                }
            }
            return _index.TryGetValue(key, out var c) ? c : null;
        }
    }
    /// &lt;summary&gt;
    /// Runtime lookup helper (CampusAudio mirror). Consumers MUST treat null as
    /// "audio missing -&gt; stay silent" so headless checks stay green on any
    /// project state (catalog absent, clip pruned, name typo). Stem constants
    /// are the single consumption contract both sides code against - they
    /// match the cloudF-B-AUDIO-B637 live-queue ids one-to-one.
    /// &lt;/summary&gt;
    public static class ScrewAudio
    {
        // cloudF-B-AUDIO-B637 stems (audio book G04 row): the loopable hook
        // BGM bed plus the three core feedback one-shots.
        public const string HookBgm = "g04_hook_bgm";
        public const string SfxUnscrewPop = "g04_sfx_unscrew_pop";
        public const string SfxCascadeTopple = "g04_sfx_cascade_topple";
        public const string SfxMagnetYank = "g04_sfx_magnet_yank";
        static ScrewAudioCatalog _catalog;
        public static ScrewAudioCatalog Catalog
        {
            get
            {
                if (_catalog == null) _catalog = Resources.Load&lt;ScrewAudioCatalog&gt;("ScrewAudioCatalog");
                return _catalog;
            }
        }
        public static AudioClip Get(string key)
        {
            var c = Catalog;
            return c != null ? c.Get(key) : null;
        }
        /// &lt;summary&gt;True when the stem is landed and playable - the X027
        /// "use art when present" branch point: the unscrew pop beat falls
```

```
        /// back to the procedural ClickSynth placeholder while false.&lt;/summary&gt;
        public static bool Has(string key)
        {
            return Get(key) != null;
        }
        /// &lt;summary&gt;
        /// Loop the requested stem on a fresh host object and return the source.
        /// Missing clip resolves to silence (returns null) - never an error, so
        /// the playtest slice runs with or without the audio wave landed.
        /// &lt;/summary&gt;
        public static AudioSource PlayLoop(string key, float volume = 0.6f)
        {
            var clip = Get(key);
            if (clip == null) return null;
            var go = new GameObject("ScrewAudio(" + key + ")");
            var src = go.AddComponent&lt;AudioSource&gt;();
            src.clip = clip;
            src.loop = true;
            src.volume = volume;
            src.Play();
            return src;
        }
        /// &lt;summary&gt;
        /// Fire a one-shot feedback stem. Missing clip = silence (no object
        /// spawned, no error) so event-driven wiring is safe pre-arrival.
        /// &lt;/summary&gt;
        public static void PlayOneShot(string key, float volume = 0.8f)
        {
            var clip = Get(key);
            if (clip == null) return;
            var go = new GameObject("ScrewSfx(" + key + ")");
            var src = go.AddComponent&lt;AudioSource&gt;();
            src.clip = clip;
            src.loop = false;
            src.volume = volume;
            src.Play();
            // static class has no UnityEngine.Object inheritance face, the
            // destroy call must be fully qualified (X924 non-MonoBehaviour law).
            UnityEngine.Object.Destroy(go, clip.length + 0.05f);
        }
    }
}
// ScrewAudioHost.cs
// ScrewAudioHost.cs - S04_Main runtime hook for the generated-audio face
// (G04 audio wiring prep, batch B643). Mirror of CampusAudioHost /
// FarmAudioHost: the scene-side face that starts the looping hook BGM bed
// on play start. ScrewAudio.PlayLoop is null-safe by contract (missing
// catalog/clip = silence, never an error), so the headless gates stay green
// whatever the import state is - wired by ProjectSetup.EnsureAudioHost
// (G13 B583 idempotent law).
```

```
using UnityEngine;
namespace Sy.G04
{
    public sealed class ScrewAudioHost : MonoBehaviour
    {
        void Start()
        {
            ScrewAudio.PlayLoop(ScrewAudio.HookBgm);
        }
    }
}
// ScrewTapHost.cs
// ScrewTapHost.cs - S04_Main scene host for the tap-twist core feel (G04-T005).
// Presentation shell over TwistEngine: one tap resolves through the engine, then all
// feedback fires inside the same frame - click sfx, screw pop-fly, board slide
// (red line 11 same-beat promise). Headless checks drive TapScrew directly, so no
// gate ever depends on input or play mode. T008 adds the magnet rescue offer:
// a discovered deadlock shows the category-gated CTA on the very same frame.
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
namespace Sy.G04
{
    public sealed class ScrewTapHost : MonoBehaviour
    {
        BoardStack stack;
        TwistEngine engine;
        BoardView view;
        TextMesh counter;
        TextMesh dirToggle;
        TextMesh reverseHint;
        TextMesh magnetOffer;
        TextMesh hintBtn;
        TextMesh skipBtn;
        readonly HashSet&lt;int&gt; popped = new HashSet&lt;int&gt;();
        readonly HashSet&lt;int&gt; countedFreed = new HashSet&lt;int&gt;();
        bool boardCleared;
        readonly List&lt;string&gt; adFunnel = new List&lt;string&gt;();
        readonly AdCapPolicy adCap = Boot.BootServices.AdCap;
        bool init;
        float toggleFlashT;
        bool magnetBudgetOk;
        bool magnetArmed;
        ScrewData hintTarget;
        TextMesh shareBtn;
        string shareChannel;
        readonly Sy.Social.ShareCardService shareComposer = new Sy.Social.ShareCardService();
        // T021 hook-signal state: sweat gag (first laugh), collapse slow-mo (first wow),
        // flawless-clear banner (first flex) - all lazy-built, zero init-time children
```

```
// so every frozen 25-child init assertion stays byte-stable.
TextMesh sweat;
TextMesh slowMoTag;
TextMesh perfectCard;
int tightenBeats;     // every tighten is a "stuck" beat - flawless clears have zero
int tightenStreak;    // consecutive tightens with no progress in between
bool assistUsed;      // magnet/hint/skip completed - assisted clears are not flawless
/// &lt;summary&gt;Magnet rescue point id (T008): draws from the shared rescue 4/day
/// category budget - the Knit Out main-revenue position per the pitch doc.&lt;/summary&gt;
public const string MagnetRescuePoint = "g04_magnet_rescue";
/// &lt;summary&gt;Hint point id (T009): draws from its own 3/day g04_hint category
/// (pitch doc "hint 3/day") and lights up the next movable screw.&lt;/summary&gt;
public const string HintPoint = "g04_hint";
/// &lt;summary&gt;Skip point id (T009): the direct take - pitch doc "skip 1/day
/// (direct category)" - shares the direct 1/day budget; the whole board falls.&lt;/summary&gt;
public const string SkipPoint = "g04_skip";
static readonly Color AccentColor = new Color(0.25f, 0.75f, 0.64f, 1f);
static readonly Color DimColor = new Color(0.55f, 0.55f, 0.55f, 1f);
static readonly Color WarmColor = new Color(0.9f, 0.45f, 0.2f, 1f);
public TwistEngine Engine { get { return engine; } }
public BoardStack Stack { get { return stack; } }
public IReadOnlyList&lt;string&gt; AdFunnel { get { return adFunnel; } }
public bool MagnetArmed { get { return magnetArmed; } }
public bool MagnetBudgetOk { get { return magnetBudgetOk; } }
public ScrewData HintTarget { get { return hintTarget; } }
/// &lt;summary&gt;Pending share channel (T010): "topple" (wow face), "help" (rescue
/// face) or "perfect" (T021 flex face); null until the first spectacular beat
/// arms the CTA.&lt;/summary&gt;
public string ShareChannel { get { return shareChannel; } }
/// &lt;summary&gt;T021 first-laugh signal: sweat gag beats fired; FirstLaughAt is the
/// sim time of the first one (-1 until then), WipeBrowShown marks the brow-wipe
/// escalation punchline. Gate-face reads, set by TightenFeedback.&lt;/summary&gt;
public int LaughBeats { get; private set; }
public double FirstLaughAt { get; private set; } = -1.0;
public bool WipeBrowShown { get; private set; }
/// &lt;summary&gt;T021 first-flex signal: true when the board cleared with zero
/// tightens and zero assists - the flawless run.&lt;/summary&gt;
public bool PerfectClear { get; private set; }
/// &lt;summary&gt;Live category prechecks (T009): always-fresh budget faces for the
/// hint/skip buttons - computed on read so precheck and consume can never
/// disagree (same policy instance as the registered ads service).&lt;/summary&gt;
public bool HintBudgetOk
{
    get { return adCap.HasCategoryBudget(AdCapPolicy.TodayKey(), AdCategories.G04Hint); }
}
public bool SkipBudgetOk
{
    get { return adCap.HasCategoryBudget(AdCapPolicy.TodayKey(), AdCategories.CategoryOfPoint(SkipPoint)); }
}
/// &lt;summary&gt;Currently selected twist direction (T007): +1 naive CW default, -1 counter.
```

```
/// FlipTwist changes it; after a reverse solve the host restores the naive default -
/// mirroring the E4 naive-first bot that solved 15/15 reverse encounters.&lt;/summary&gt;
public int TwistDir = +1;
void Start() { EnsureInit(); }
/// &lt;summary&gt;Builds the opening board (or a check-injected custom stack), the view
/// and HUD. Idempotent so checks can call it directly.&lt;/summary&gt;
public void EnsureInit(BoardStack customStack = null)
{
    if (init) return;
    init = true;
    stack = customStack != null ? customStack : TwistEngine.BuildOpening();
    SaveWire.LoadOrDefault();   // T013: meta progression first - local authority, zero visuals
    engine = new TwistEngine(stack);
    view = new BoardView(transform);
    view.Rebuild(stack);
    ClickSynth.EnsureSource(this);
    BuildHud();
    // T023 device matrix: fit the scene camera to the frozen design frame
    // (orthographic, safe-area letterbox). Null-safe in headless gates.
    CameraFit.ApplyToMainCamera();
    // T017 art wiring: full-frame workshop backdrop (S04 face) through the
    // shared helper - also the S00 boot beat's first-frame face.
    EnsureWorkshopBackdrop();
    // B654 AIGC label duty (FE-20260924-B-11): the TJ AI-assisted art
    // assets in the catalog flip the explicit AI-content label rules on.
    EnsureAigcLabel();
    if (ServiceLocator.TryGet&lt;IAdsService&gt;(out var ads))
        ads.OnAdEvent += OnAdLifecycle;   // evt_ad funnel (T008), rescue point only
    // T014: share-return count - a cold boot through a friend's share link
    // counts one local return on the boot beat (zero-server: the platform
    // only hands us the raw string, every parse stays on-device).
    var suite = Boot.BootServices.Suite;
    ILaunchInfo launchInfo = suite != null ? suite.Launch : null;
    LaunchQuery launchQuery = launchInfo != null ? launchInfo.GetLaunchQuery() : null;
    if (ShareLoop.CountShareReturn(launchQuery, out string returnChannel))
        adFunnel.Add("evt_share_return_" + (returnChannel ?? "plain") + "@" + engine.SimTime.ToString("0.00"));
    engine.ScanDeadlock();   // an all-jammed layout offers from tick zero
    ProcessEngineEvents(0, null);   // the init-time offer shows in that same beat
}
void BuildHud()
{
    MakeText("Caption", "One screw out, the board falls.", new Vector3(0f, 2.7f, -1f), 34);
    counter = MakeText("Counter", "freed 0", new Vector3(0f, -2.7f, -1f), 28);
    dirToggle = MakeText("DirToggle", "TWIST CW", new Vector3(-2.35f, -2.7f, -1f), 26);
    reverseHint = MakeText("ReverseHint", "", new Vector3(0f, 2.25f, -1f), 24);
    reverseHint.color = new Color(1f, 0.45f, 0.2f, 1f);   // warm signal for the betrayal beat
    // T009 ad points: persistent CTAs, bottom-right stack - far below the board
    // (screws live within |y| &lt;= 0.94) and clear of the dir toggle / magnet offer.
    hintBtn = MakeText("HintBtn", "HINT", new Vector3(2.35f, -2.7f, -1f), 26);
    skipBtn = MakeText("SkipBtn", "SKIP", new Vector3(2.35f, -2.2f, -1f), 22);
```

```
    // T017 art wiring: catalog icons beside the two persistent ad CTAs
    // (null-safe: no catalog art = text-only, the procedural state).
    HangIcon(hintBtn, "ui_icon_hint", -0.75f, 0f, 0.45f);
    HangIcon(skipBtn, "ui_icon_skip", -0.75f, 0f, 0.4f);
}
/// &lt;summary&gt;TextMesh world-size normalization (T023 device matrix): the
/// engine renders TextMesh at roughly fontSize/10 world units, so a 34pt
/// caption measures 45.8 world units wide - clipped off every phone. The
/// 0.1 scale lands each band back inside the frozen design frame while
/// keeping the authored font ratios (hit boxes stay generous by design).&lt;/summary&gt;
const float HudTextScale = 0.1f;
TextMesh MakeText(string childName, string text, Vector3 localPos, int size)
{
    var go = new GameObject(childName);
    go.transform.SetParent(transform, false);
    go.transform.localPosition = localPos;
    go.transform.localScale = Vector3.one * HudTextScale;
    var tm = go.AddComponent&lt;TextMesh&gt;();
    tm.font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
    tm.text = text;
    tm.fontSize = size;
    tm.anchor = TextAnchor.MiddleCenter;
    tm.color = Color.white;
    return tm;
}
/// &lt;summary&gt;Standalone workshop backdrop (T017 art wiring): a root-level
/// SpriteRenderer behind the z=0 board plane, camera-relative, scaled to
/// cover the frozen 7.2x6.4 design frame. Root object - NOT a host child -
/// so the frozen 25-child opening-layout canary never sees it. Null-safe:
/// no catalog art = no backdrop (the camera clear color stays the
/// procedural state). Shared by the S04 host init and the S00 boot beat,
/// so the very first rendered frame already shows the game world.&lt;/summary&gt;
public static GameObject EnsureWorkshopBackdrop()
{
    Sprite bg = ArtRuntime.Get("bg_workshop");
    if (bg == null) return null;
    var bgGo = new GameObject("WorkshopBackdrop");
    Camera cam = Camera.main;
    Vector3 camPos = cam != null ? cam.transform.position : Vector3.zero;
    bgGo.transform.position = new Vector3(camPos.x, camPos.y, 5f);   // behind the z=0 board plane
    var bgSr = bgGo.AddComponent&lt;SpriteRenderer&gt;();
    bgSr.sprite = bg;
    bgSr.sortingOrder = -100;
    float cover = Mathf.Max(7.2f / bg.bounds.size.x, 6.4f / bg.bounds.size.y);
    bgGo.transform.localScale = new Vector3(cover, cover, 1f);
    return bgGo;
}
/// &lt;summary&gt;The explicit AI-content disclosure line (B654, FE-20260924-B-11):
/// the TJ AI-assisted art assets in the catalog (T017 legs, 9+7 pieces) flip
/// the 2025-09-01 AI-content label rules on. Root-level footer - NOT a host
```

```
/// child - so the frozen 25-child opening-layout canary never sees it
/// (workshop-backdrop law); persistent, camera-relative, parked in the
/// 7.2x6.4 design frame's bottom strip below every HUD row.&lt;/summary&gt;
public const string AigcLineText = "VISUAL ART ASSETS IN THIS GAME ARE AI-ASSISTED";
/// &lt;summary&gt;Live footer instance (root object; idempotent; Unity nulls it
/// on scene switches so every scene rebuilds its own).&lt;/summary&gt;
public static GameObject AigcLabel;
/// &lt;summary&gt;Builds (or returns) the persistent AIGC disclosure footer. The
/// art-only wording ships today; once the four TJ audio stems land and get
/// wired the line upgrades to the art+audio version - one at a time.&lt;/summary&gt;
public static GameObject EnsureAigcLabel()
{
    if (AigcLabel != null) return AigcLabel;
    var go = new GameObject("AigcLabel");
    Camera cam = Camera.main;
    Vector3 camPos = cam != null ? cam.transform.position : Vector3.zero;
    go.transform.position = new Vector3(camPos.x, camPos.y - 3.05f, -1f);
    go.transform.localScale = Vector3.one * HudTextScale;
    var tm = go.AddComponent&lt;TextMesh&gt;();
    tm.font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
    tm.text = AigcLineText;
    tm.fontSize = 16;
    tm.anchor = TextAnchor.MiddleCenter;
    tm.color = new Color(0.78f, 0.80f, 0.83f, 1f);   // quiet steel: legible on wood and wash alike
    AigcLabel = go;
    return go;
}
/// &lt;summary&gt;HUD icon hung on a text button's GameObject (T017 art wiring):
/// world-space offset beside/above the label, size given in world units so
/// the icon never depends on the 0.1 HUD text scale. Art-driven enhancement -
/// a null lookup renders nothing, so the text-only procedural state stays
/// byte-identical and every frozen layout gate holds.&lt;/summary&gt;
static void HangIcon(TextMesh button, string stem, float worldX, float worldY, float worldSize)
{
    if (button == null) return;
    Sprite art = ArtRuntime.Get(stem);
    if (art == null) return;   // X027: no art = keep the text-only state
    var go = new GameObject(stem);
    go.transform.SetParent(button.transform, false);
    Vector3 p = button.transform.position;
    go.transform.position = new Vector3(p.x + worldX, p.y + worldY, p.z + 0.01f);
    float native = art.bounds.size.x;                 // world units at scale 1
    float lossy = button.transform.lossyScale.x;      // 0.1 on the HUD texts
    float s = native &gt; 0f &amp;&amp; lossy &gt; 0f ? worldSize / (native * lossy) : 1f;
    go.transform.localScale = new Vector3(s, s, 1f);
    var sr = go.AddComponent&lt;SpriteRenderer&gt;();
    sr.sprite = art;
    sr.sortingOrder = 310;   // above every plate/screw, below the sweat gag band
}
void Update()
```

```
{
    if (!init) return;
    if (toggleFlashT &gt; 0f)
    {
        toggleFlashT -= Time.deltaTime;
        if (toggleFlashT &lt;= 0f &amp;&amp; dirToggle != null)
        {
            dirToggle.color = Color.white;
            dirToggle.text = TwistDir &gt; 0 ? "TWIST CW" : "TWIST CCW";
        }
    }
    if (Input.GetMouseButtonDown(0)) TryTapAt(Input.mousePosition);
    else if (Input.touchCount &gt; 0 &amp;&amp; Input.GetTouch(0).phase == TouchPhase.Began)
        TryTapAt(Input.GetTouch(0).position);
}
void TryTapAt(Vector3 screenPos)
{
    Vector3 world = WorldPoint(screenPos);
    if (HitToggle(world)) { FlipTwist(); return; }
    if (HitOffer(world)) { TryRequestMagnet(); return; }
    if (HitTextButton(hintBtn, world, 0.7f)) { _ = RequestHint(); return; }
    if (HitTextButton(skipBtn, world, 0.7f)) { _ = RequestSkip(); return; }
    if (HitTextButton(shareBtn, world, 1.15f)) { TapShareCta(); return; }
    if (magnetArmed)
    {
        ScrewData m = HitScrewAny(world);
        if (m != null) MagnetYank(m);
        return;
    }
    ScrewData s = HitScrew(world);
    if (s != null) TapScrew(s);
}
/// &lt;summary&gt;Hit test for a HUD text button (T009): a generous rect around its
/// text. Null or hidden buttons are untouchable.&lt;/summary&gt;
static bool HitTextButton(TextMesh button, Vector3 world, float halfWidth)
{
    if (button == null || !button.gameObject.activeSelf) return false;
    Vector3 p = button.transform.position;
    return Mathf.Abs(world.x - p.x) &lt;= halfWidth &amp;&amp; Mathf.Abs(world.y - p.y) &lt;= 0.3f;
}
/// &lt;summary&gt;Hit test for the magnet rescue offer (T008): a generous rect around
/// its text. Absent or hidden offer is untouchable.&lt;/summary&gt;
bool HitOffer(Vector3 world)
{
    if (magnetOffer == null || !magnetOffer.gameObject.activeSelf) return false;
    Vector3 p = magnetOffer.transform.position;
    return Mathf.Abs(world.x - p.x) &lt;= 1.45f &amp;&amp; Mathf.Abs(world.y - p.y) &lt;= 0.3f;
}
/// &lt;summary&gt;Armed magnet mode (T008): the yank target can be any unfreed screw,
/// rusted and covered included - freed screws stay untouchable.&lt;/summary&gt;
```

```
ScrewData HitScrewAny(Vector3 worldPos)
{
    ScrewData best = null;
    float bestD = 0.55f * 0.55f;
    for (int i = 0; i &lt; stack.Screws.Count; i++)
    {
        ScrewData s = stack.Screws[i];
        if (s.freed) continue;
        Vector3 d = BoardView.ScrewLocalPos(s, stack) - worldPos;
        float q = d.x * d.x + d.y * d.y;
        if (q &lt;= bestD) { bestD = q; best = s; }
    }
    return best;
}
void TryRequestMagnet()
{
    if (!magnetBudgetOk || magnetArmed) return;
    _ = RequestMagnetRescue();   // mock ads resolve inline; real SDKs await off-frame
}
/// &lt;summary&gt;Hit test for the direction toggle (T007): a generous rect around its text.&lt;/summary&gt;
bool HitToggle(Vector3 world)
{
    if (dirToggle == null) return false;
    Vector3 p = dirToggle.transform.position;
    return Mathf.Abs(world.x - p.x) &lt;= 0.85f &amp;&amp; Mathf.Abs(world.y - p.y) &lt;= 0.3f;
}
/// &lt;summary&gt;Flips the selected twist direction and refreshes the toggle label.&lt;/summary&gt;
public void FlipTwist()
{
    TwistDir = -TwistDir;
    toggleFlashT = 0f;
    if (dirToggle != null)
    {
        dirToggle.text = TwistDir &gt; 0 ? "TWIST CW" : "TWIST CCW";
        dirToggle.color = Color.white;
    }
}
Vector3 WorldPoint(Vector3 screenPos)
{
    Camera cam = Camera.main;
    if (cam == null) return Vector3.zero;
    float dist = -cam.transform.position.z;   // camera looks at the z=0 board plane
    Vector3 p = cam.ScreenToWorldPoint(new Vector3(screenPos.x, screenPos.y, dist));
    p.z = 0f;
    return p;
}
/// &lt;summary&gt;Finds the nearest tappable screw under the world point (no physics: cell math).&lt;/summary&g
ScrewData HitScrew(Vector3 worldPos)
{
    ScrewData best = null;
```

```
    float bestD = 0.55f * 0.55f;   // hit radius matches the rendered screw size
    for (int i = 0; i &lt; stack.Screws.Count; i++)
    {
        ScrewData s = stack.Screws[i];
        if (s.freed || s.covered || s.rusted) continue;
        Vector3 d = BoardView.ScrewLocalPos(s, stack) - worldPos;
        float q = d.x * d.x + d.y * d.y;
        if (q &lt;= bestD) { bestD = q; best = s; }
    }
    return best;
}
/// &lt;summary&gt;Default tap uses the selected twist direction (T007).&lt;/summary&gt;
public void TapScrew(ScrewData s) { TapScrew(s, TwistDir); }
/// &lt;summary&gt;Resolves one tap through the engine and fires every feedback in the same frame.&lt;/summary&g
public void TapScrew(ScrewData s, int dir)
{
    int before = engine.Events.Count;
    TapOutcome outcome = engine.Tap(s, dir);
    if (outcome == TapOutcome.Loosened || outcome == TapOutcome.Freed)
        tightenStreak = 0;   // T021: progress clears the sweat gag
    ProcessEngineEvents(before, s);
}
/// &lt;summary&gt;Fires host feedback for every engine event appended after the
/// given index. Shared by taps, magnet yanks and the init-time offer scan.&lt;/summary&gt;
void ProcessEngineEvents(int before, ScrewData s)
{
    for (int i = before; i &lt; engine.Events.Count; i++)
    {
        FeelEvent e = engine.Events[i];
        switch (e.type)
        {
            case "screw_pop": PopScrew(s); break;
            case "count_up": SetCounter(); break;
            case "board_slide": BoardLeaves(ParseLayer(e.detail), false); break;
            case "cascade_collapse": EngageSlowMo(); break;   // T021 wow: screws tumbling with the slide
            case "board_topple": BoardLeaves(ParseLayer(e.detail), true); ArmShareCta("topple"); EngageSlowMo();
            case "chain_collapse": BoardLeaves(ParseLayer(e.detail), true); ArmShareCta("topple"); EngageSlowMo(
            case "reverse_tighten": TightenFeedback(s, true); break;
            case "screw_tighten": TightenFeedback(s, false); break;
            case "reverse_solved": ReverseSolvedFeedback(); break;
            case "deadlock_offer": ShowMagnetOffer(); ArmShareCta("help"); break;
            case "magnet_pull": MagnetPulledBeat(); break;
            case "tap_blocked": BlockedNudge(); break;
            default: break;
        }
    }
    // T013: every freed screw enters the codex exactly once - all free
    // paths (tap, magnet yank, skip sweep) land here, deduped by id - and
    // a fully cleared board closes the level. Pure data, no beats, no HUD.
    for (int i = 0; i &lt; stack.Screws.Count; i++)
```

```
        {
            ScrewData f = stack.Screws[i];
            if (f.freed &amp;&amp; !countedFreed.Contains(f.id))
            {
                countedFreed.Add(f.id);
                SaveWire.OnFreed(f);
            }
        }
        if (stack.Remaining == 0 &amp;&amp; !boardCleared)
        {
            boardCleared = true;
            // T030: the clear beat carries its flawless verdict into the career save (U138 F5)
            SaveWire.OnLevelCleared(tightenBeats == 0 &amp;&amp; !assistUsed);
            TryPerfectBeat();   // T021: a zero-jam zero-assist clear is the flex beat
        }
    }
    /// &lt;summary&gt;Category-gated rescue request (T008): the offer was pre-checked when
    /// it showed; this call consumes the rescue 4/day category through the shared
    /// policy instance and arms the magnet on completion.&lt;/summary&gt;
    public async System.Threading.Tasks.Task&lt;bool&gt; RequestMagnetRescue()
    {
        if (!magnetBudgetOk || magnetArmed) return false;
        if (!ServiceLocator.TryGet&lt;IAdsService&gt;(out var ads)) return false;
        RewardResult r = await ads.ShowRewarded(MagnetRescuePoint);
        if (r != null &amp;&amp; r.Completed)
        {
            magnetArmed = true;
            assistUsed = true;   // T021: a rescue-assisted clear is not flawless
            SaveWire.OnAdConsumed(MagnetRescuePoint);   // T013: daily counter, same boundary as the cap
            if (magnetOffer != null) magnetOffer.text = "MAGNET ARMED - TAP ANY SCREW";
            return true;
        }
        if (magnetOffer != null) magnetOffer.text = "AD SKIPPED - TAP TO RETRY";
        return false;
    }
    /// &lt;summary&gt;Applies the armed magnet (T008): yanks the chosen screw through the
    /// engine (rescue discipline: needs the pending offer) and fires every feedback
    /// in the same frame - pop, click, counter, offer-out.&lt;/summary&gt;
    public void MagnetYank(ScrewData s)
    {
        if (!magnetArmed || s == null || s.freed) return;
        magnetArmed = false;
        int before = engine.Events.Count;
        if (!engine.MagnetPull(s)) return;
        ProcessEngineEvents(before, s);
        if (magnetOffer != null) magnetOffer.gameObject.SetActive(false);
    }
    /// &lt;summary&gt;Category-gated hint request (T009): pre-checks the g04_hint 3/day
    /// budget AND that a movable screw exists before promising anything (the §
    /// gate - a deadlocked board has nothing to hint and must not burn the ad);
```

```
/// on completion the next movable screw lights up in the accent color.&lt;/summary&gt;
public async System.Threading.Tasks.Task&lt;bool&gt; RequestHint()
{
    if (!HintBudgetOk)
    {
        adFunnel.Add("evt_ad_offer_capped@" + engine.SimTime.ToString("0.00"));
        SetButton(hintBtn, "HINT - DAILY LIMIT", DimColor);
        return false;
    }
    ScrewData target = engine.FindHintScrew(TwistDir);
    if (target == null) { SetButton(hintBtn, "NOTHING TO HINT", DimColor); return false; }
    if (!ServiceLocator.TryGet&lt;IAdsService&gt;(out var ads)) return false;
    adFunnel.Add("evt_ad_offer@" + engine.SimTime.ToString("0.00"));
    RewardResult r = await ads.ShowRewarded(HintPoint);
    if (r != null &amp;&amp; r.Completed)
    {
        SaveWire.OnAdConsumed(HintPoint);   // T013: daily counter, same boundary as the cap
        assistUsed = true;   // T021: a hint-assisted clear is not flawless
        hintTarget = target;
        var go = view.FindChild("screw_" + target.id);
        if (go != null &amp;&amp; go.GetComponent&lt;HintFx&gt;() == null) go.AddComponent&lt;HintFx&gt;();
        SetButton(hintBtn, "HINTED - FOLLOW THE GLOW", AccentColor);
        return true;
    }
    SetButton(hintBtn, "HINT - AD SKIPPED, TAP TO RETRY", WarmColor);
    return false;
}
/// &lt;summary&gt;Category-gated skip request (T009): the direct take - pre-checks the
/// direct 1/day budget and a non-cleared board, then on completion the whole
/// stack topples house-of-cards style through the engine on one beat and every
/// collapse feedback fires in the same frame. A visible rescue offer hides with
/// the level: the jam dies with the board.&lt;/summary&gt;
public async System.Threading.Tasks.Task&lt;bool&gt; RequestSkip()
{
    if (!SkipBudgetOk)
    {
        adFunnel.Add("evt_ad_offer_capped@" + engine.SimTime.ToString("0.00"));
        SetButton(skipBtn, "SKIP - DAILY LIMIT", DimColor);
        return false;
    }
    if (stack.Remaining == 0) { SetButton(skipBtn, "BOARD ALREADY CLEAR", DimColor); return false; }
    if (!ServiceLocator.TryGet&lt;IAdsService&gt;(out var ads)) return false;
    adFunnel.Add("evt_ad_offer@" + engine.SimTime.ToString("0.00"));
    RewardResult r = await ads.ShowRewarded(SkipPoint);
    if (r != null &amp;&amp; r.Completed)
    {
        SaveWire.OnAdConsumed(SkipPoint);   // T013: the ad burned = the daily take burned (cap parity)
        assistUsed = true;   // T021: a skip-swept clear is not flawless
        int before = engine.Events.Count;
        if (!engine.SkipLevel()) return false;   // cleared between offer and completion
```

```
        ProcessEngineEvents(before, null);
        if (magnetOffer != null) magnetOffer.gameObject.SetActive(false);
        SetButton(skipBtn, "SKIPPED!", AccentColor);
        return true;
    }
    SetButton(skipBtn, "SKIP - AD SKIPPED, TAP TO RETRY", WarmColor);
    return false;
}
/// &lt;summary&gt;Sets a HUD text button's label and color (persistent states, like
/// the magnet offer's own text states - no timers to desync).&lt;/summary&gt;
static void SetButton(TextMesh button, string text, Color color)
{
    if (button == null) return;
    button.text = text;
    button.color = color;
}
/// &lt;summary&gt;Share CTA (T010): lazily built on the first spectacular beat
/// (topple/chain) or the first deadlock offer, so the opening layout stays
/// byte-identical for every frozen init assertion. The pending channel picks
/// the card face - the wow collapse vs the friend-rescue hook.&lt;/summary&gt;
void ArmShareCta(string channel)
{
    // Already showing this face: no state change, no duplicate funnel line
    // (a multi-board topple fires one CTA show, not one per board).
    if (shareBtn != null &amp;&amp; shareBtn.gameObject.activeSelf &amp;&amp; shareChannel == channel) return;
    shareChannel = channel;
    if (shareBtn == null) shareBtn = MakeText("ShareCta", "", new Vector3(0f, 1.35f, -1f), 26);
    shareBtn.gameObject.SetActive(true);
    shareBtn.text = channel == "help" ? "STUCK? ASK A FRIEND"
        : channel == "perfect" ? "SHARE THE FLAWLESS RUN" : "SHARE THE FALL";
    shareBtn.color = channel == "help" ? new Color(0.9f, 0.45f, 0.2f, 1f)
        : channel == "perfect" ? new Color(1f, 0.82f, 0.25f, 1f) : new Color(0.25f, 0.75f, 0.64f, 1f);
    adFunnel.Add("evt_share_cta_" + channel + "@" + engine.SimTime.ToString("0.00"));
}
/// &lt;summary&gt;Player tapped the share CTA: compose the pending channel's card
/// through the shared RT pipeline, then hand it to the channel suite as a
/// SharePayload (T014) - the composed PNG path rides the payload so real
/// channel suites attach the card image. ReadPixels is a sync stall, so
/// composition only ever runs on the player's own tap frame - never on an
/// engine beat.&lt;/summary&gt;
public void TapShareCta()
{
    Sy.Social.ComposeResult res = ComposeShareCard();
    if (res.Ok) SendSharePayload(res.Path);
    SetButton(shareBtn, res.Ok ? "CARD SHARED!" : "CARD FAILED - TAP TO RETRY",
        res.Ok ? new Color(0.25f, 0.75f, 0.64f, 1f) : new Color(0.55f, 0.55f, 0.55f, 1f));
}
/// &lt;summary&gt;Hands the composed card to the channel suite (T014): builds the
/// SharePayload from the pending channel's card spec - the T010 copy query
/// plus the formation codec - and calls the platform Share face. Editor and
```

```
/// batchmode record it in MockPlatformService.Shares for gate assertions; a
/// null suite or platform is a silent no-op so every headless run stays green.&lt;/summary&gt;
void SendSharePayload(string cardPath)
{
    var suite = Boot.BootServices.Suite;
    if (suite == null || suite.Platform == null) return;
    Sy.Social.ShareCard card = BuildShareCard();
    SharePayload payload = card == null ? null : ShareLoop.PayloadFor(card, cardPath);
    if (payload == null) return;
    suite.Platform.Share(payload);
    adFunnel.Add("evt_share_platform_" + shareChannel + "@" + engine.SimTime.ToString("0.00"));
}
/// &lt;summary&gt;Composes the pending channel's card (T010 dual-channel face) and
/// records the compose outcome in the funnel. With no context armed yet the
/// result is a machine-readable fail - never a crash.&lt;/summary&gt;
public Sy.Social.ComposeResult ComposeShareCard()
{
    Sy.Social.ShareCard card = BuildShareCard();
    if (card == null)
        return new Sy.Social.ComposeResult { Ok = false, FailReason = "no_share_context" };
    Sy.Social.ComposeResult res = shareComposer.Compose(card);
    adFunnel.Add("evt_share_" + shareChannel + (res.Ok ? "_compose_ok" : "_compose_fail")
        + "@" + engine.SimTime.ToString("0.00"));
    return res;
}
/// &lt;summary&gt;Builds the pending channel's card spec (T010 face); null when
/// no share context is armed yet. The payload builder and the compose path
/// read the very same spec, so what the platform sends is what the card
/// showed.&lt;/summary&gt;
public Sy.Social.ShareCard BuildShareCard()
{
    if (string.IsNullOrEmpty(shareChannel)) return null;
    if (shareChannel == "help") return ShareCards.HelpCard(stack);
    if (shareChannel == "perfect") return ShareCards.PerfectCard(stack);
    return ShareCards.ToppleCard(stack);
}
/// &lt;summary&gt;Deadlock offer (T008): pre-checks the rescue category budget before
/// anything is promised (§  gate), then shows the CTA on the frame the jam was
/// discovered. Lazy-built so the opening layout stays byte-identical.&lt;/summary&gt;
void ShowMagnetOffer()
{
    magnetBudgetOk = adCap.HasCategoryBudget(AdCapPolicy.TodayKey(), AdCategories.Rescue);
    if (magnetOffer == null)
    {
        magnetOffer = MakeText("MagnetOffer", "", new Vector3(0f, -1.9f, -1f), 26);
        // T017 art wiring: the rescue offer carries its art faces - the
        // magnet unit above the text, the button badge beside it
        // (null-safe: text-only stays the procedural state).
        HangIcon(magnetOffer, "magnet_rescue", 0f, 0.45f, 0.7f);
        HangIcon(magnetOffer, "ui_icon_magnet_btn", -2.1f, 0f, 0.4f);
```

```
    }
    magnetOffer.gameObject.SetActive(true);
    magnetOffer.text = magnetBudgetOk ? "MAGNET RESCUE - TAP TO YANK ANY SCREW" : "MAGNET RESCUE - DAILY BUDGET
    magnetOffer.color = magnetBudgetOk ? new Color(0.25f, 0.75f, 0.64f, 1f) : new Color(0.55f, 0.55f, 0.55f, 1f)
    adFunnel.Add("evt_ad_offer" + (magnetBudgetOk ? "" : "_capped") + "@" + engine.SimTime.ToString("0.00"));
}
/// &lt;summary&gt;Magnet yank beat: the offer flashes the accent color as the screw
/// leaves - then hides (MagnetYank), so the rescue reads as one visible act.&lt;/summary&gt;
void MagnetPulledBeat()
{
    if (magnetOffer != null)
    {
        magnetOffer.text = "YANKED!";
        magnetOffer.color = new Color(0.25f, 0.75f, 0.64f, 1f);
    }
    // B643: the yank beat carries the generated magnet sfx (null-safe
    // silence pre-arrival).
    ScrewAudio.PlayOneShot(ScrewAudio.SfxMagnetYank);
}
/// &lt;summary&gt;A blocked tap (rusted/covered screw) pulses the magnet offer when
/// one is up, nudging the rescue; without an offer the beat stays silent exactly
/// as before T008 (zero regression on the frozen paths).&lt;/summary&gt;
void BlockedNudge()
{
    if (magnetOffer != null &amp;&amp; magnetOffer.gameObject.activeSelf)
        magnetOffer.color = new Color(0.9f, 0.45f, 0.2f, 1f);
}
/// &lt;summary&gt;evt_ad funnel (T008/T009): lifecycle events of every G04 ad point
/// (magnet rescue / hint / skip) land in the host funnel list for gate
/// assertions; S2 promotes this face to the unified telemetry pipeline.&lt;/summary&gt;
void OnAdLifecycle(AdLifecycle e)
{
    if (e.PointId != MagnetRescuePoint &amp;&amp; e.PointId != HintPoint &amp;&amp; e.PointId != SkipPoint) retu
    adFunnel.Add("evt_ad_" + e.Type.ToString().ToLowerInvariant() + "@" + engine.SimTime.ToString("0.00"));
}
/// &lt;summary&gt;Wrong-direction beat (T007): the screw shivers tighter, its thread mark
/// pulses, the toggle flashes - and the first reverse betrayal shows the teach hint.&lt;/summary&gt;
void TightenFeedback(ScrewData s, bool teach)
{
    tightenBeats++;          // T021: a tighten is a stuck beat - flawless clears have zero
    tightenStreak++;
    ShowSweat(s);            // T021 first-laugh signal: the wrench sweats
    var go = view.FindChild("screw_" + s.id);
    if (go != null &amp;&amp; go.GetComponent&lt;TightenFx&gt;() == null) go.AddComponent&lt;TightenFx&gt;();
    if (dirToggle != null)
    {
        // Store-shot clarity: the stale direction label reads as contradictory
        // copy in a still frame ("reverse" next to a bold CW) - say the state.
        dirToggle.text = "WRONG WAY!";
        dirToggle.color = new Color(1f, 0.35f, 0.25f);
```

```
        toggleFlashT = 0.6f;
    }
    if (teach &amp;&amp; reverseHint != null &amp;&amp; reverseHint.text.Length == 0)
        reverseHint.text = "REVERSE THREAD - TWIST CCW!";
}
/// &lt;summary&gt;Reverse mastered: clear the hint and restore the naive default so the
/// next screw starts from the instinct direction (E4 naive-first bot parity).&lt;/summary&gt;
void ReverseSolvedFeedback()
{
    if (reverseHint != null) reverseHint.text = "";
    if (TwistDir &lt; 0) FlipTwist();
}
/// &lt;summary&gt;T021 first-laugh signal: the wrench sweats over the tightened screw -
/// one drop on the first jam, the brow-wipe punchline when the streak builds (any
/// progress resets it, see TapScrew). Reuses one lazy child so the gag never
/// allocates a new object mid-play.&lt;/summary&gt;
void ShowSweat(ScrewData s)
{
    if (sweat == null)
    {
        sweat = MakeText("Sweat", "", new Vector3(0f, 0.55f, -1f), 18);
        // B410: HUD texts default to sortingOrder 0, so the sweat gag
        // over an L1 screw hid behind the upper plates and only a glyph
        // sliver peeked past the board edge (round-1 review note).
        // Draw the first-laugh signal above every plate so the beat
        // is always visible (red line 11: the laugh must be felt).
        sweat.GetComponent&lt;MeshRenderer&gt;().sortingOrder = 320;
        sweat.gameObject.SetActive(false);
    }
    sweat.gameObject.SetActive(true);
    sweat.text = tightenStreak &gt;= 2 ? "*wipes brow*" : "sweat drop...";
    sweat.color = tightenStreak &gt;= 2 ? new Color(0.85f, 0.9f, 1f, 1f) : Color.white;
    Vector3 p = BoardView.ScrewLocalPos(s, stack);
    sweat.transform.localPosition = new Vector3(p.x, p.y + 0.55f, -1f);
    SweatFx fx = sweat.GetComponent&lt;SweatFx&gt;();
    if (fx == null) sweat.gameObject.AddComponent&lt;SweatFx&gt;();
    else fx.Relight();
    if (tightenStreak &gt;= 2) WipeBrowShown = true;
    LaughBeats++;
    if (FirstLaughAt &lt; 0.0) FirstLaughAt = engine.SimTime;
}
/// &lt;summary&gt;T021 first-wow signal: the collapse beats (cascade / topple / chain)
/// get a short global slow-mo window plus the SLOW-MO tag - the player has the
/// beat to screen-record the fall (the topple share card is already armed).&lt;/summary&gt;
void EngageSlowMo()
{
    if (slowMoTag == null)
    {
        slowMoTag = MakeText("SlowMoTag", "SLOW-MO - SCREEN IT!", new Vector3(0f, 1.9f, -1f), 22);
        slowMoTag.color = new Color(0.25f, 0.75f, 0.64f, 1f);
```

```
            slowMoTag.gameObject.SetActive(false);
        }
        slowMoTag.gameObject.SetActive(true);
        SlowMoFx.Engage(gameObject, slowMoTag);
        // B643: the collapse beats carry the generated topple sfx (null-safe
        // silence pre-arrival - no procedural fallback exists for this beat).
        ScrewAudio.PlayOneShot(ScrewAudio.SfxCascadeTopple);
    }
    /// &lt;summary&gt;T021 first-flex signal: a flawless clear - zero tightens, zero
    /// assists - flips the PERFECT banner and re-arms the share CTA on the
    /// flawless-run card (the flex beat outranks the wow beat when both were earned).&lt;/summary&gt;
    void TryPerfectBeat()
    {
        if (tightenBeats &gt; 0 || assistUsed) return;
        PerfectClear = true;
        if (perfectCard == null)
            perfectCard = MakeText("PerfectCard", "FLAWLESS! ZERO-JAM CLEAR", new Vector3(0f, 1.75f, -1f), 30);
        perfectCard.gameObject.SetActive(true);
        perfectCard.color = new Color(1f, 0.82f, 0.25f, 1f);
        ArmShareCta("perfect");
        adFunnel.Add("evt_signal_perfect@" + engine.SimTime.ToString("0.00"));
    }
    void PopScrew(ScrewData s)
    {
        popped.Add(s.id);
        var go = view.FindChild("screw_" + s.id);
        if (go != null &amp;&amp; go.GetComponent&lt;PopFx&gt;() == null) go.AddComponent&lt;PopFx&gt;();
        // X027 audio law (B643): the generated unscrew pop owns the beat
        // once the cloudF wave lands; until then the procedural ka-da
        // placeholder keeps the click (U020 two-tier rule).
        if (ScrewAudio.Has(ScrewAudio.SfxUnscrewPop)) ScrewAudio.PlayOneShot(ScrewAudio.SfxUnscrewPop);
        else ClickSynth.PlayClick();
        view.BuildHole(s, stack);   // T017: the freed cell reads as a hole (null-safe no-op without art)
    }
    void SetCounter()
    {
        if (counter != null) counter.text = "freed " + stack.FreedTotal;
    }
    /// &lt;summary&gt;Feedback for a board leaving the field: a corner slide, or the slow
    /// house-of-cards topple (T006 visual hammer - the heavier stagger reads as slow-mo).&lt;/summary&gt;
    void BoardLeaves(int layer, bool topple)
    {
        var boardGo = view.FindChild("board_" + layer);
        if (boardGo != null)
        {
            if (topple &amp;&amp; boardGo.GetComponent&lt;ToppleFx&gt;() == null) boardGo.AddComponent&lt;ToppleFx&g
            else if (!topple &amp;&amp; boardGo.GetComponent&lt;SlideFx&gt;() == null) boardGo.AddComponent&lt;Slide
        }
        // screws carried away tumble right after the board = the cascade visual
        float stagger = topple ? 0.06f : 0.04f;
```

```
            var onLayer = stack.ScrewsOnLayer(layer);
            for (int i = 0; i &lt; onLayer.Count; i++)
            {
                ScrewData s = onLayer[i];
                if (!s.freed || popped.Contains(s.id)) continue;
                var go = view.FindChild("screw_" + s.id);
                if (go == null || go.GetComponent&lt;PopFx&gt;() != null) continue;
                PopFx fx = go.AddComponent&lt;PopFx&gt;();
                fx.delay = stagger;
                stagger += topple ? 0.06f : 0.04f;
            }
            if (topple) SetCounter();   // the freed count soars through the tumble, count it live
        }
        /// &lt;summary&gt;Parses the layer number out of the E4-frozen "layer N corner slides open" detail.&lt;/summary
        static int ParseLayer(string detail)
        {
            int at = detail.IndexOf("layer ");
            if (at &lt; 0) return -1;
            int start = at + 6;
            int end = detail.IndexOf(' ', start);
            if (end &lt; 0) end = detail.Length;
            int layer;
            return int.TryParse(detail.Substring(start, end - start), out layer) ? layer : -1;
        }
    }
}
// ShareCards.cs
// ShareCards.cs - dual-channel share card specs (G04-T010).
// Both channels ride the shared Sy.Social composer (P02 T015b RT pipeline,
// X032 zero-server local-template edition): the wow channel turns the
// house-of-cards collapse into a braggable card, the help channel turns the
// deadlock into the friend-rescue hook. Specs are pure state reads - the host
// composes only on the player's own share tap (ReadPixels is a sync stall,
// never runs on an engine beat).
using Sy.Social;
namespace Sy.G04
{
    public static class ShareCards
    {
        public const string WowTitle = "THE WHOLE BOARD FELL!";
        public const string HelpTitle = "NOT EVEN A GOD CAN UNSCREW THIS";
        public const string WowFooter = "ScrewOut - one screw out, the board falls.";
        public const string HelpFooter = "ScrewOut - help a friend out.";
        public const string PerfectTitle = "FLAWLESS DISASSEMBLY!";
        public const string PerfectFooter = "ScrewOut - zero jams, pure skill.";
        /// &lt;summary&gt;Wow channel spec: the spectacular collapse beat (topple/chain),
        /// the recordable "look what the game did" moment.&lt;/summary&gt;
        public static ShareCard ToppleCard(BoardStack stack)
        {
            var card = AutoCardComposer.Card("share_card_collapse", WowTitle, new[]
```

```
        {
            "BOARDS DOWN: " + stack.SlidBoards + "/" + stack.Boards.Count,
            "SCREWS FREED: " + stack.FreedTotal,
            "CAN YOU CLEAR IT CLEANER?"
        }, WowFooter, "src=share&amp;ch=topple&amp;boards=" + stack.SlidBoards + "&amp;freed=" + stack.FreedTotal);
        card.Formation = SnapshotFormation(stack, out int cols, out int rows);
        card.FormationCols = cols;
        card.FormationRows = rows;
        return card;
    }
    /// &lt;summary&gt;Perfect channel spec (T021 flex signal): the zero-jam, zero-assist
    /// clear - the bragging-rights card. Pure skill flex, the strongest share hook
    /// a clean player can send.&lt;/summary&gt;
    public static ShareCard PerfectCard(BoardStack stack)
    {
        var card = AutoCardComposer.Card(null, PerfectTitle, new[]
        {
            "ZERO JAMS - NOT ONE TIGHTEN",
            "SCREWS FREED: " + stack.FreedTotal,
            "CAN YOU CLEAR IT CLEAN?"
        }, PerfectFooter, "src=share&amp;ch=perfect&amp;freed=" + stack.FreedTotal);
        card.Formation = SnapshotFormation(stack, out int cols, out int rows);
        card.FormationCols = cols;
        card.FormationRows = rows;
        return card;
    }
    /// &lt;summary&gt;Help channel spec: the deadlock rescue hook (X032 peer-gift
    /// position - free, no ad gate, pure organic spread).&lt;/summary&gt;
    public static ShareCard HelpCard(BoardStack stack)
    {
        var card = AutoCardComposer.Card("share_card_help", HelpTitle, new[]
        {
            "I AM STUCK WITH " + stack.Remaining + " SCREWS",
            "THE BOARD JAMMED ON ME",
            "SEND ME A RESCUE LINE"
        }, HelpFooter, "src=share&amp;ch=help&amp;remaining=" + stack.Remaining);
        card.Formation = SnapshotFormation(stack, out int cols, out int rows);
        card.FormationCols = cols;
        card.FormationRows = rows;
        return card;
    }
    /// &lt;summary&gt;Board snapshot for the card formation layer: one row per board
    /// (top board on the top row), one cell per grid slot in the first cell row.
    /// Tier law: freed=gold 5, covered=white 1, reverse=green 2, rusted=blue 3,
    /// plain jam=purple 4, empty slot=0 - the squint-glance reads the jam shape.&lt;/summary&gt;
    public static CardCell[] SnapshotFormation(BoardStack stack, out int cols, out int rows)
    {
        rows = stack.Boards.Count;
        cols = 1;
        for (int i = 0; i &lt; rows; i++)
```

```
                if (stack.Boards[i].width &gt; cols) cols = stack.Boards[i].width;
            var cells = new CardCell[rows * cols];
            for (int r = 0; r &lt; rows; r++)
            {
                int layer = rows - 1 - r;   // formation row 0 renders at the top
                for (int c = 0; c &lt; cols; c++)
                {
                    ScrewData s = stack.ScrewAt(layer, c, 0);
                    cells[r * cols + c] = new CardCell { Tier = s == null ? 0 : TierOf(s), Stack = 1 };
                }
            }
            return cells;
        }
        /// &lt;summary&gt;Screw state to card tier (deterministic, gate-assertable).&lt;/summary&gt;
        public static int TierOf(ScrewData s)
        {
            if (s.freed) return 5;
            if (s.covered) return 1;
            if (s.rusted) return 3;
            if (s.reverse) return 2;
            return 4;
        }
    }
}
// ShareLoop.cs
// ShareLoop.cs - platform share wiring and launch-query return counting (G04-T014).
// The T010 card specs stay byte-stable; this loop is the transport face: the
// composed card rides the channel suite as a SharePayload whose query carries
// the formation codec, so a friend's client replays the exact jam shape locally
// (X032 zero-server replay-code form - the platform only transports the string,
// every parse and derive stays on-device). Launch-query counting is local too:
// a cold boot through a share link counts one return for the host funnel.
// No cloud face anywhere in the loop (red line 12).
using System;
using Sy.Platform;
using Sy.Social;
namespace Sy.G04
{
    public static class ShareLoop
    {
        public const string SrcKey = "src";
        public const string ChannelKey = "ch";
        public const string FormationKey = "form";
        public const string SrcShare = "share";
        /// &lt;summary&gt;Local share-return counter (zero-server: the count never leaves the device).&lt;/summary&gt;
        public static int ShareReturns;
        /// &lt;summary&gt;Builds the platform share payload from a card spec: the T010
        /// copy query plus the formation codec, the composed PNG path as the
        /// image. Null card or payload stays a caller-side no-op, never a throw.&lt;/summary&gt;
        public static SharePayload PayloadFor(ShareCard card, string imagePath)
```

```
{
    if (card == null) return null;
    string query = card.Query ?? "";
    if (card.HasFormation)
    {
        string form = EncodeFormation(card.Formation, card.FormationCols, card.FormationRows);
        if (form.Length &gt; 0) query = query + "&amp;" + FormationKey + "=" + form;
    }
    return new SharePayload { Title = card.Title, ImageUrl = imagePath, Query = query };
}
/// &lt;summary&gt;Formation codec: "RxC" + one tier digit per cell, row-major
/// with row 0 first (the card snapshot order). Tier clamps to 0-5; the
/// G04 snapshot never stacks a cell, so the codec carries tiers only.&lt;/summary&gt;
public static string EncodeFormation(CardCell[] cells, int cols, int rows)
{
    if (cells == null || cols &lt; 1 || rows &lt; 1 || cells.Length != cols * rows) return "";
    var sb = new System.Text.StringBuilder();
    sb.Append(rows).Append('x').Append(cols).Append(':');
    for (int i = 0; i &lt; cells.Length; i++)
    {
        int tier = cells[i].Tier;
        if (tier &lt; 0) tier = 0;
        if (tier &gt; 5) tier = 5;
        sb.Append((char)('0' + tier));
    }
    return sb.ToString();
}
/// &lt;summary&gt;Friend-side local recompute: replays the formation grid out
/// of a share query. False - never a throw - on absent or malformed
/// form data, so any hostile or stale query degrades to a no-op.&lt;/summary&gt;
public static bool TryReplayFormation(string query, out CardCell[] cells, out int cols, out int rows)
{
    cells = null;
    cols = 0;
    rows = 0;
    string form = ParamOf(query, FormationKey);
    if (string.IsNullOrEmpty(form)) return false;
    int colon = form.IndexOf(':');
    if (colon &lt; 3 || colon + 1 &gt;= form.Length) return false;
    string dims = form.Substring(0, colon);
    int sep = dims.IndexOf('x');
    if (sep &lt;= 0 || sep + 1 &gt;= dims.Length) return false;
    if (!int.TryParse(dims.Substring(0, sep), out rows) || rows &lt; 1 || rows &gt; 64) return false;
    if (!int.TryParse(dims.Substring(sep + 1), out cols) || cols &lt; 1 || cols &gt; 64) return false;
    string digits = form.Substring(colon + 1);
    if (digits.Length != rows * cols) return false;
    var grid = new CardCell[rows * cols];
    for (int i = 0; i &lt; digits.Length; i++)
    {
        int tier = digits[i] - '0';
```

```
                if (tier &lt; 0 || tier &gt; 5) return false;
                grid[i] = new CardCell { Tier = tier, Stack = 1 };
            }
            cells = grid;
            return true;
        }
        /// &lt;summary&gt;Launch-entry return count (T014): a boot query carrying
        /// src=share counts one local return and hands back the channel for
        /// the funnel line. Organic opens and foreign sources never count.&lt;/summary&gt;
        public static bool CountShareReturn(LaunchQuery q, out string channel)
        {
            channel = null;
            if (q == null || string.IsNullOrEmpty(q.Raw)) return false;
            if (!SrcShare.Equals(ParamOf(q.Raw, SrcKey), StringComparison.Ordinal)) return false;
            channel = ParamOf(q.Raw, ChannelKey);
            ShareReturns++;
            return true;
        }
        /// &lt;summary&gt;Test hygiene: resets the local return counter.&lt;/summary&gt;
        public static void Reset() { ShareReturns = 0; }
        /// &lt;summary&gt;Reads one k=v pair out of an &amp;-joined query. Null when the
        /// key is absent. Values never contain '&amp;' in this codec.&lt;/summary&gt;
        public static string ParamOf(string query, string key)
        {
            if (string.IsNullOrEmpty(query) || string.IsNullOrEmpty(key)) return null;
            int at = 0;
            while (at &lt;= query.Length)
            {
                int end = query.IndexOf('&amp;', at);
                string pair = end &lt; 0 ? query.Substring(at) : query.Substring(at, end - at);
                int eq = pair.IndexOf('=');
                if (eq &gt; 0 &amp;&amp; pair.Substring(0, eq) == key) return eq + 1 &lt; pair.Length ? pair.Substring(e
                if (end &lt; 0) break;
                at = end + 1;
            }
            return null;
        }
    }
}
// SlideFx.cs
// SlideFx.cs - board slide-away animation (G04-T005, the third same-beat feedback).
// The board slides open a corner with a slight tilt and fades out, then self-destructs.
using UnityEngine;
namespace Sy.G04
{
    public sealed class SlideFx : MonoBehaviour
    {
        public const float Life = 0.55f;
        float age;
        SpriteRenderer sr;
```

```
        Color color0;
        Vector3 pos0;
        void Awake()
        {
            sr = GetComponent&lt;SpriteRenderer&gt;();
            if (sr != null) color0 = sr.color;
            pos0 = transform.localPosition;
        }
        void Update()
        {
            age += Time.deltaTime;
            float k = Mathf.Clamp01(age / Life);
            transform.localPosition = pos0 + new Vector3(2.6f * k, -0.9f * k * k, 0f);
            transform.localRotation = Quaternion.Euler(0f, 0f, -14f * k);
            if (sr != null)
            {
                Color c = color0;
                c.a = 1f - k;
                sr.color = c;
            }
            if (age &gt;= Life) Destroy(gameObject);
        }
    }
}
// SlowMoFx.cs
// SlowMoFx.cs - collapse slow-mo beat (G04-T021 first-wow signal): the collapse
// moments (cascade / topple / chain knock) dip the global clock for a short unscaled
// window - the fall reads as a recordable slow-mo, and the SLOW-MO tag invites the
// screen grab (zero-server: the phone's own recorder captures, the share card spreads).
// The restore rides unscaled time so a frame hitch can never strand the game in slow
// motion, and Advance() lets headless gates tick the restore deterministically.
using UnityEngine;
namespace Sy.G04
{
    public sealed class SlowMoFx : MonoBehaviour
    {
        public const float Life = 0.9f;      // unscaled seconds of hero time
        public const float Factor = 0.35f;   // global clock dip during the window
        float age;
        TextMesh tagMesh;
        /// &lt;summary&gt;Engages the window on the host (idempotent: a multi-board skip or
        /// a chain of collapses is one slow-mo, not one per board).&lt;/summary&gt;
        public static SlowMoFx Engage(GameObject hostGo, TextMesh slowMoTag)
        {
            SlowMoFx fx = hostGo.GetComponent&lt;SlowMoFx&gt;();
            if (fx != null) return fx;
            fx = hostGo.AddComponent&lt;SlowMoFx&gt;();
            fx.tagMesh = slowMoTag;
            Time.timeScale = Factor;
            return fx;
```

```
        }
        void Update() { Advance(Time.unscaledDeltaTime); }
        /// &lt;summary&gt;Pure restore tick (unscaled dt) - gates drive this directly.&lt;/summary&gt;
        public void Advance(float unscaledDt)
        {
            age += unscaledDt;
            if (age &gt;= Life) Restore();
        }
        /// &lt;summary&gt;Ends the window: clock back to 1.0, tag lights out, fx self-removes.&lt;/summary&gt;
        public void Restore()
        {
            Time.timeScale = 1f;
            if (tagMesh != null) tagMesh.gameObject.SetActive(false);
            if (Application.isPlaying) Destroy(this);
            else DestroyImmediate(this);
        }
        void OnDestroy()
        {
            // Teardown safety (gate DestroyImmediate / scene unload): never strand a
            // slow clock - batchmode exits would serialize it into TimeManager.asset.
            if (Time.timeScale == Factor) Time.timeScale = 1f;
        }
    }
}
// SweatFx.cs
// SweatFx.cs - stuck-wrench sweat gag (G04-T021 first-laugh signal): a tighten beat
// lights a sweat line over the tightened screw - the wrench is working hard and it
// shows. One drop on the first jam, the brow-wipe punchline when the streak builds
// (the host swaps the text; this fx only owns the fade). Update-driven like the other
// fx so headless gates assert presence, and Advance() carries the pure fade so gates
// can tick it deterministically without play mode.
using UnityEngine;
namespace Sy.G04
{
    public sealed class SweatFx : MonoBehaviour
    {
        public const float Life = 0.6f;   // short gag: never outstays the next decision
        float age;
        TextMesh label;
        Vector3 origin;
        Color color0;
        void Awake()
        {
            label = GetComponent&lt;TextMesh&gt;();
            origin = transform.localPosition;
            if (label != null) color0 = label.color;
        }
        void Update() { Advance(Time.deltaTime); }
        /// &lt;summary&gt;Re-caches the fade anchors for a beat that re-lights a still-live
        /// fx (the host moved the label and swapped its text mid-life).&lt;/summary&gt;
```

```
        public void Relight()
        {
            age = 0f;
            origin = transform.localPosition;
            if (label != null) color0 = label.color;
        }
        /// &lt;summary&gt;Pure fade tick (dt in seconds) - gates drive this directly.&lt;/summary&gt;
        public void Advance(float dt)
        {
            age += dt;
            float k = Mathf.Clamp01(age / Life);
            if (label != null)
            {
                Color c = color0;
                c.a = 1f - k;
                label.color = c;
            }
            transform.localPosition = origin + new Vector3(0f, -0.35f * k, 0f);   // the drop slides off
            if (age &gt;= Life) Done();
        }
        void Done()
        {
            transform.localPosition = origin;
            if (label != null) label.color = color0;
            gameObject.SetActive(false);   // reuse pattern: the host re-lights it on the next beat
            if (Application.isPlaying) Destroy(this);
            else DestroyImmediate(this);
        }
    }
}
// TightenFx.cs
// TightenFx.cs - wrong-direction tap feedback (G04-T007): the screw shivers as it
// locks tighter and its thread mark pulses, teaching the player to read the mark and
// flip the twist. Update-driven like PopFx so headless checks can assert presence;
// destroys only the component - the screw stays on the board (it tightened, not freed).
using UnityEngine;
namespace Sy.G04
{
    public sealed class TightenFx : MonoBehaviour
    {
        public const float Life = 0.45f;
        float age;
        Vector3 origin;
        void Awake() { origin = transform.localPosition; }
        void Update()
        {
            age += Time.deltaTime;
            float k = age / Life;
            // decaying shiver: reads as the thread biting tighter, not a wiggle
            float amp = 0.06f * (1f - k);
```

```
            transform.localPosition = origin + new Vector3(Mathf.Sin(age * 55f) * amp, 0f, 0f);
            var mark = transform.Find("thread_mark");
            if (mark != null)
            {
                float pulse = 0.42f * (1f + 0.4f * (1f - k) * (0.5f + 0.5f * Mathf.Sin(age * 25f)));
                mark.localScale = new Vector3(pulse, pulse, 1f);
            }
            if (age &gt;= Life)
            {
                transform.localPosition = origin;
                Destroy(this);
            }
        }
    }
}
// ToppleFx.cs
// ToppleFx.cs - board house-of-cards topple animation (G04-T006, the visual hammer).
// Slower and heavier than SlideFx on purpose: the long tumble reads as the collapse
// slow-mo moment (recordable for the share card). Runs purely in Update so headless
// checks can assert the component without any play mode.
using UnityEngine;
namespace Sy.G04
{
    public sealed class ToppleFx : MonoBehaviour
    {
        public const float Life = 1.1f;   // deliberately slower than the 0.55s corner slide
        float age;
        SpriteRenderer sr;
        Color color0;
        Vector3 pos0;
        void Awake()
        {
            sr = GetComponent&lt;SpriteRenderer&gt;();
            if (sr != null) color0 = sr.color;
            pos0 = transform.localPosition;
        }
        void Update()
        {
            age += Time.deltaTime;
            float k = Mathf.Clamp01(age / Life);
            // tips over the freed anchor corner, then slides down off the field
            transform.localPosition = pos0 + new Vector3(1.1f * k, -2.2f * k * k, 0f);
            transform.localRotation = Quaternion.Euler(0f, 0f, -52f * k * k);
            if (sr != null)
            {
                Color c = color0;
                c.a = 1f - k;
                sr.color = c;
            }
            if (age &gt;= Life) Destroy(gameObject);
```

```
        }
    }
}
// TwistEngine.cs
// TwistEngine.cs - pure C# tap-twist logic over BoardStack (G04-T005).
// Byte-level port of the E4 graybox semantics: same event vocabulary, same tap cadence,
// same pop-triple-on-same-tick rule, same slide/cascade logging order. Redline11Check
// drives this deterministically; ScrewTapHost is only the presentation shell on top.
// No UnityEngine dependency - sim time is logical seconds.
using System;
using System.Collections.Generic;
namespace Sy.G04
{
    public enum TapOutcome { None, Loosened, Freed, Tightened, Blocked }
    public sealed class FeelEvent
    {
        public double t;
        public string type;
        public string detail;
        public FeelEvent(double t, string type, string detail)
        {
            this.t = t;
            this.type = type;
            this.detail = detail;
        }
    }
    /// &lt;summary&gt;
    /// Drives taps on a BoardStack and records the feel timeline as events.
    /// E4-frozen promises (Hook5): caption 0.20s / first pop triple 0.50s /
    /// board slide 1.50s / first cascade 1.50s / reverse teach 1.85s.
    /// &lt;/summary&gt;
    public sealed class TwistEngine
    {
        public const double TapCadenceSec = 0.35;   // sim seconds per tap (player cadence)
        public const int ChainKnockMax = 2;         // a board beneath with &lt;= this many screws gets swept
        public double SimTime;
        public readonly List&lt;FeelEvent&gt; Events = new List&lt;FeelEvent&gt;();
        public readonly BoardStack Stack;
        public int Taps, WrongTaps, Cascades, SlidBoards, ReverseEncountered, ReverseSolved;
        public int ToppleBoards, ChainKnocked, Collapses;   // collapse discoveries of every kind (T006)
        public int Deadlocks, Rescues;                       // magnet rescue window stats (T008, E3-3)
        public int Skips;                                    // level skips taken through the direct-take point (T009)
        bool _offerPending;                                 // one offer per deadlock, cleared by the pull
        public TwistEngine(BoardStack stack) { Stack = stack; }
        public void SetTime(double t) { SimTime = t; }
        public void Log(string type, string detail) { Events.Add(new FeelEvent(SimTime, type, detail)); }
        public double TimeOf(string type)
        {
            for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) return Events[i].t;
            return -1.0;
```

```
}
public bool EventExistsAt(string type, double t)
{
    for (int i = 0; i &lt; Events.Count; i++)
        if (Events[i].type == type &amp;&amp; Math.Abs(Events[i].t - t) &lt; 1e-9) return true;
    return false;
}
/// &lt;summary&gt;Taps a screw with a chosen direction (+1 naive default, -1 counter).&lt;/summary&gt;
public TapOutcome Tap(ScrewData s, int dir)
{
    if (s == null || s.freed) return TapOutcome.None;
    if (s.covered || s.rusted) { Log("tap_blocked", "screw " + s.id); return TapOutcome.Blocked; }
    Taps++;
    SimTime += TapCadenceSec;   // feedback fires on the same tick as the tap (red line 11)
    bool correct = s.reverse ? dir == -1 : dir == +1;
    if (correct)
    {
        s.progress++;
        if (s.progress &gt;= s.neededTaps)
        {
            FreeByTap(s, true);
            ScanDeadlock();   // a slide can uncover an all-rusted layer: offer same tick (T008)
            return TapOutcome.Freed;
        }
        return TapOutcome.Loosened;
    }
    s.wrongTaps++;
    WrongTaps++;
    if (s.reverse)
    {
        if (s.wrongTaps == 1) ReverseEncountered++;
        Log("reverse_tighten", "screw " + s.id + " tightened! check the thread mark");
    }
    else
    {
        // a normal screw tapped in the flipped direction gets the same physical
        // feedback (T007): silent tighten would read as a broken tap
        Log("screw_tighten", "screw " + s.id + " tightened - flip the twist back");
    }
    return TapOutcome.Tightened;
}
/// &lt;summary&gt;Shared free path for taps and magnet yanks. byTap=false (magnet)
/// suppresses the reverse-solved stat: the screw left without the player
/// ever mastering its direction, so it is a bypass, not a solve (T008).&lt;/summary&gt;
void FreeByTap(ScrewData s, bool byTap)
{
    BoardLayerData b = Stack.Boards[s.layer];
    // count the screws that would tumble away with a slide, before the model mutates
    int fellWithBoard = 0;
    if (b.freed + 1 &gt;= b.slideAt &amp;&amp; !b.slid)
```

```
    {
        var onLayer = Stack.ScrewsOnLayer(s.layer);
        for (int i = 0; i &lt; onLayer.Count; i++)
            if (!onLayer[i].freed &amp;&amp; onLayer[i] != s) fellWithBoard++;
    }
    int freedAtPop = Stack.FreedTotal + 1;   // pop-moment count, before any carried-away frees
    Stack.FreeScrew(s);   // BoardStack owns the slide/coverage state machine
    // first-delight triple: pop + click + count-up, all on the same sim tick
    Log("screw_pop", "screw " + s.id + " flies out");
    Log("sfx_click", "ka-da");
    Log("count_up", "freed=" + freedAtPop);
    if (s.reverse &amp;&amp; byTap) { ReverseSolved++; Log("reverse_solved", "screw " + s.id + " direction maste
    if (b.slid)
    {
        SlidBoards++;
        Log("board_slide", "layer " + b.layer + " corner slides open");
        if (fellWithBoard &gt;= 2)
        {
            Cascades++;
            Collapses++;
            Log("cascade_collapse", fellWithBoard + " screws tumble down with the board");
        }
        return;
    }
    if (!s.key) return;
    // key anchor pulled before the slide threshold: the whole board topples house-of-cards
    // style and the impact can sweep the nearly-empty board beneath - the chain reveal is
    // the E3-1 signature moment, the anti "Nth screw-twist" differentiator (T006).
    int tumbled = Stack.ToppleBoard(s.layer);
    ToppleBoards++;
    Collapses++;
    SlidBoards++;
    Log("board_topple", "layer " + s.layer + " topples house-of-cards, " + tumbled + " screws tumble down");
    int next = s.layer + 1;
    while (next &lt; Stack.Boards.Count &amp;&amp; !Stack.Boards[next].slid &amp;&amp; RemainingOn(next) &lt;= C
    {
        int swept = Stack.ToppleBoard(next);
        ChainKnocked++;
        Collapses++;
        SlidBoards++;
        Log("chain_collapse", "layer " + next + " knocked flat by the falling board, " + swept + " screws swept"
        next++;
    }
}
int RemainingOn(int layer)
{
    int n = 0;
    var onLayer = Stack.ScrewsOnLayer(layer);
    for (int i = 0; i &lt; onLayer.Count; i++) if (!onLayer[i].freed) n++;
    return n;
```

```
}
/// &lt;summary&gt;A board is deadlocked while screws remain but none of them can
/// ever be tapped free: every remaining screw is rusted or covered (T008).&lt;/summary&gt;
public bool IsDeadlocked()
{
    bool anyLeft = false;
    for (int i = 0; i &lt; Stack.Screws.Count; i++)
    {
        ScrewData s = Stack.Screws[i];
        if (s.freed) continue;
        anyLeft = true;
        if (!s.covered &amp;&amp; !s.rusted) return false;
    }
    return anyLeft;
}
/// &lt;summary&gt;Deadlock scan: fires the rescue offer on the same sim tick the
/// deadlock appears on (E4 offer-same-beat promise - never hard-stuck without
/// an offer). One offer per deadlock; MagnetPull clears the pending flag so a
/// later jam offers again (E3-3 rescue window 1-5 per 200 taps).&lt;/summary&gt;
public bool ScanDeadlock()
{
    if (_offerPending || !IsDeadlocked()) return false;
    _offerPending = true;
    Deadlocks++;
    Log("deadlock_offer", "board jammed - magnet rescue offered");
    return true;
}
/// &lt;summary&gt;Magnet rescue (T008): yanks any one screw straight out - even a
/// rusted or covered one - breaking the jam the offer fired for. The yank
/// routes through the same free path as a tap, so a key anchor still topples
/// its board: the rescue is a real choice, not a free pass. Requires the
/// pending offer (rescue discipline: no free magnet outside a deadlock).&lt;/summary&gt;
public bool MagnetPull(ScrewData s)
{
    if (s == null || s.freed || !_offerPending) return false;
    _offerPending = false;
    Rescues++;
    SimTime += TapCadenceSec;   // the yank costs one beat, same as a tap
    Log("magnet_pull", "screw " + s.id + " yanked out by the magnet");
    FreeByTap(s, false);
    ScanDeadlock();   // the yank itself can reveal another all-rusted layer
    return true;
}
/// &lt;summary&gt;Hint candidate (T009): the next movable screw - unfreed, uncovered,
/// unrusted. Prefers one whose correct twist matches the chosen direction (real
/// progress with the current twist), falls back to any movable screw when the
/// direction has no match (the T007 thread marks teach the flip for free).
/// Id order keeps the choice deterministic; null when nothing is movable
/// (deadlocked or cleared) so callers can precheck before promising anything.&lt;/summary&gt;
public ScrewData FindHintScrew(int dir)
```

```
{
    ScrewData fallback = null;
    for (int i = 0; i &lt; Stack.Screws.Count; i++)
    {
        ScrewData s = Stack.Screws[i];
        if (s.freed || s.covered || s.rusted) continue;
        if ((s.reverse ? -1 : +1) == dir) return s;
        if (fallback == null) fallback = s;
    }
    return fallback;
}
/// &lt;summary&gt;Level skip (T009): the direct-take point - every remaining board
/// topples house-of-cards style on one beat, so the skip reads as the most
/// spectacular collapse in the game instead of a fade to win. The pending
/// deadlock offer clears: the jam ends with the level. Refuses a cleared
/// board; the topples count as real collapses because they really happened.&lt;/summary&gt;
public bool SkipLevel()
{
    if (Stack.Remaining == 0) return false;
    _offerPending = false;
    Skips++;
    SimTime += TapCadenceSec;   // one beat, same as a magnet yank
    Log("level_skip", "skip taken - the whole board falls");
    for (int layer = 0; layer &lt; Stack.Boards.Count; layer++)
    {
        BoardLayerData b = Stack.Boards[layer];
        if (b.slid) continue;
        int tumbled = Stack.ToppleBoard(layer);
        ToppleBoards++;
        Collapses++;
        SlidBoards++;
        Log("board_topple", "layer " + layer + " topples house-of-cards, " + tumbled + " screws tumble down");
    }
    Log("count_up", "freed=" + Stack.FreedTotal);   // the freed counter soars with the fall
    return true;
}
// ---- Opening layout (port of GrayboxSim.BuildOpening, cell-based, T006 third board) ----
// Layer0: 6 screws, slides at 3 freed -&gt; 3 carried away = first cascade (teach collapse).
// Layer1: 8 screws, first one is the reverse KEY anchor with 2 needed taps: the betrayal
//         screw doubles as the house-of-cards trigger, pulling it topples the whole board.
// Layer2: 2 loose screws on a thin plate, too light to survive the impact - the chain
//         sweeps it, so one teaching game shows all three collapse discoveries.
public static BoardStack BuildOpening()
{
    var stack = new BoardStack();
    stack.AddBoard(4, 2, 3);
    for (int i = 0; i &lt; 6; i++)
    {
        int cellX = i % 3;
        int cellY = i / 3;
```

```
                stack.AddScrew(0, cellX, cellY, false, false, 1);
            }
            stack.AddBoard(4, 2, 6);
            for (int i = 0; i &lt; 8; i++)
            {
                int cellX = i % 4;
                int cellY = i / 4;
                bool reverse = i == 0 || i % 4 == 2;   // idx 0 (2 taps, key) + idx 2 + idx 6
                int neededTaps = i == 0 ? 2 : 1;
                ScrewData anchor = stack.AddScrew(1, cellX, cellY, reverse, false, neededTaps);
                anchor.key = i == 0;
            }
            stack.AddBoard(4, 2, 2);
            stack.AddScrew(2, 1, 0, false, false, 1);
            stack.AddScrew(2, 2, 0, false, false, 1);
            return stack;
        }
    }
}
```